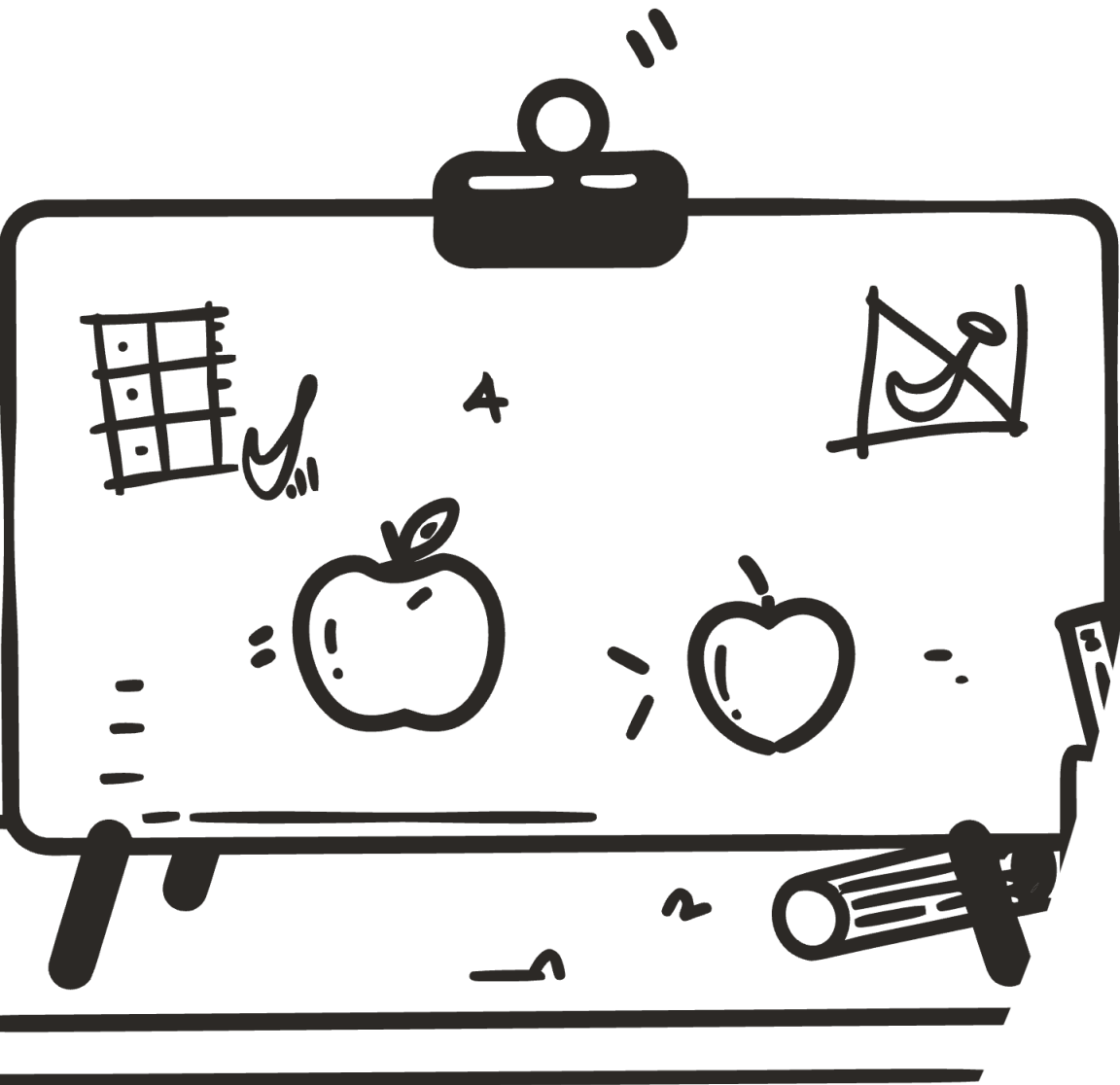


AI 비기너를 위한 머신러닝과 딥러닝 핵심

아주 얇고 넓게 AI의 핵심 개념 이해하기



머신러닝의 시작

지도학습

정답 데이터로 학습

- KNN 분류기
- 선형회귀
- 로지스틱 회귀

비지도학습

정답 없이 패턴 발견

- K-평균 군집화
- PCA 차원축소

강화학습

보상 최대화 학습

- 환경 상호작용
- 정책 최적화

데이터 세트

- 훈련 데이터 세트로 훈련하고 검증 데이터 세트로 검증하면서 모델 학습
- 학습된 모델을 테스트 데이터 세트로 성능 확인

과대적합(overfitting) vs 과소적합(underfitting)

- 과대적합: 훈련점수는 좋으나 실전점수는 낮음
- 과소적합: 훈련점수보다 오히려 실전 점수가 더 높음(훈련이 부족한 경우)

사이킷런(Scikit-learn)

- 대표적인 오픈 소스 머신러닝 라이브러리
- 모델 선택 -> 훈련(fit) -> 예측(predict)의 동일한 패턴 함수 제공

KNN 분류알고리즘

K-Nearest Neighbors

핵심 원리

가장 가까운 K개 이웃의 다수결로 분류

01

거리 계산

새 데이터와 모든 훈련 데이터 간
거리 측정

03

다수결 투표

K개 중 최다 클래스로 예측

* 클래스란 ? 객체를 추상화한 것. 고양이라는 클래스를 분류하기 위해 수염길이, 귀 색깔, 눈 크기 등의 데이터를 학습

02

이웃 선택

가장 가까운 K개 선택
K=5가 일반적 기본값

주의사항

☐ 데이터 전처리 필수

특성 간 스케일 차이가 크면 표준점수화 필요
(데이터 - 평균) / 표준편차

선형회귀와 다항회귀

머신러닝에서 회귀는 연속적인 값을 예측하는 과정으로 이해

1

선형회귀

$$y = ax + b$$

직선으로 근사 예측

예) 거리 = 가중치 * 속도 + 시작위치

2

다항회귀

$$y = ax^2 + bx + c$$

곡선으로 더 정확한 예측

예) 거리 = 가중치1 * 속도제곱 + 가중치2 * 속도 + 시작위치

3

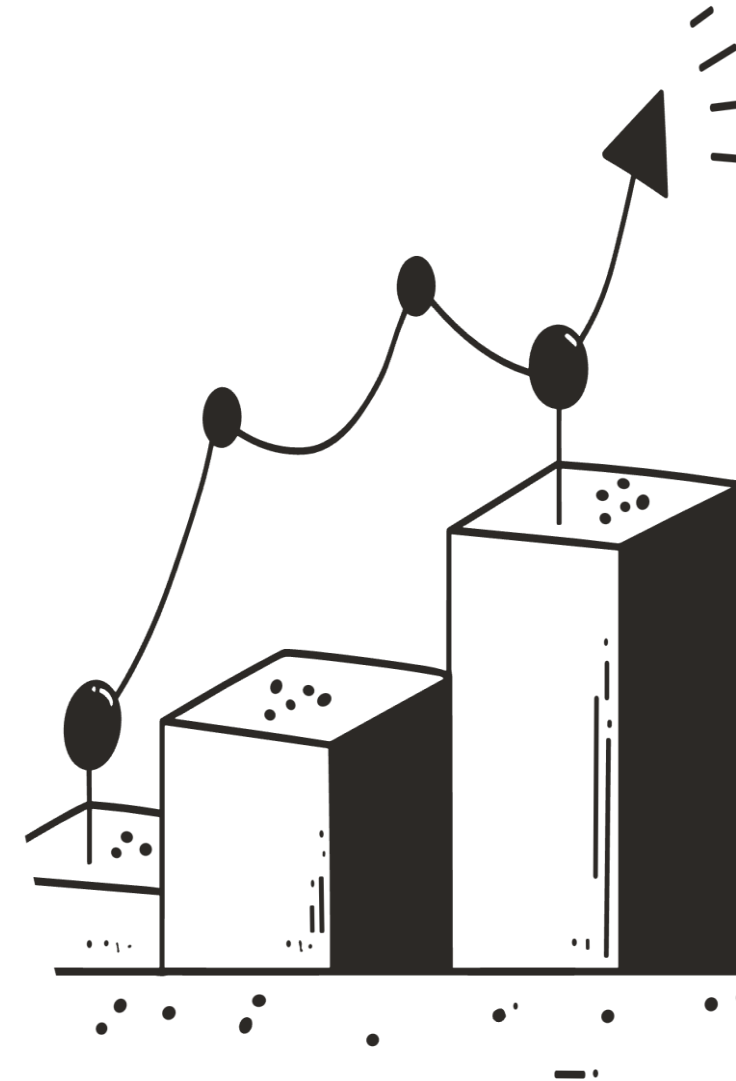
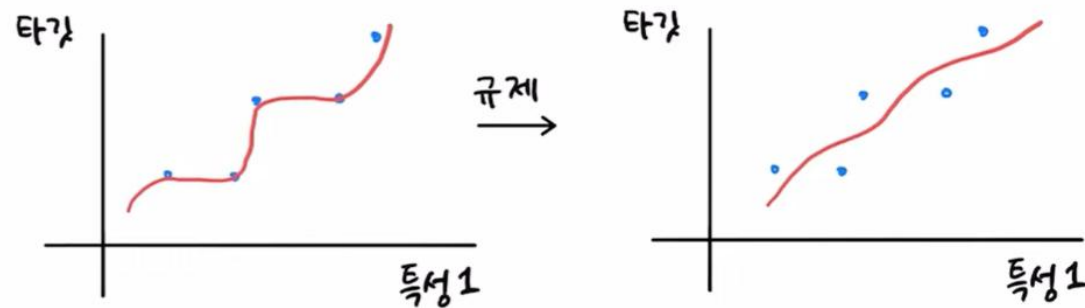
특성공학

$$Y = w_1x_1 + w_2x_2 + \dots + b$$

복수개의 특성을 이용한 선형회귀

과대적합 방지를 위해 규제 적용 필요

- 리지(Ridge): 계수 제곱합 사용
- 라쏘(Lasso): 계수 절대값 사용



로지스틱 회귀

선형회귀는 연속적인 특정 값을 정확하게 예측하는 기법이라면 로지스틱 회귀는 클래스(범주)에 속할 확률을 예측하는 기법임

시그모이드 함수

$$\sigma(z) = 1 / (1 + e^{-z})$$

0~1 사이 확률값 출력

이항분류

양성/음성 두 클래스 분류

0.5 기준 판단: 이상 양성, 이하 음성 클래스

다항분류

여러 클래스 확률 계산

가장 높은 확률 선택

$$z = w_1x_1 + w_2x_2 + \dots + b$$

$$y = 1 / (1 + e^{(-z)}) \leftarrow \text{Sigmoid 함수}$$

* e는 자연상수로 약 2.718

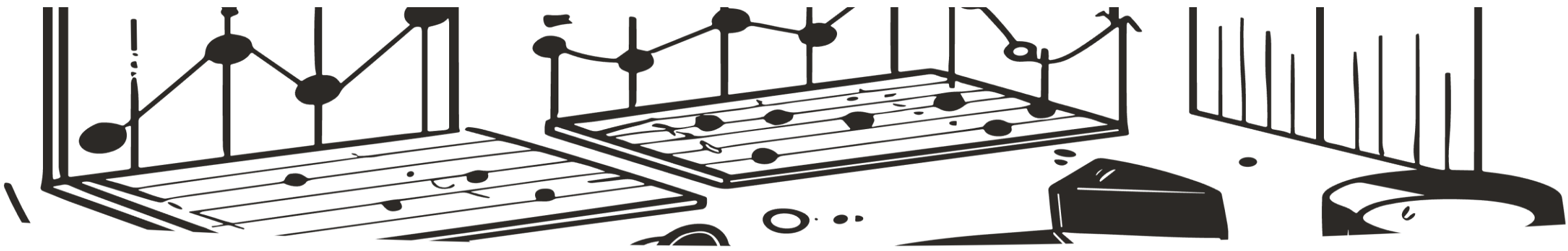
* σ 는 시그마로 표준편차나 시그모이드 함수를 나타냄

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(C=20, max_iter=1000)
lr.fit(train_scaled, train_target)
proba = lr.predict_proba(test_scaled[:5])
print(np.round(proba, decimals=3))
```

[	[0.	0.014	0.842	0.	0.135	0.007	0.003]
[	0.	0.003	0.044	0.	0.007	0.946	0.]
[	0.	0.	0.034	0.934	0.015	0.016	0.]
[	0.011	0.034	0.305	0.006	0.567	0.	0.076]
[	0.	0.	0.904	0.002	0.089	0.002	0.001]]

첫번째는 약 84%로 개, 두번째는 약 95%로 고양이, ...

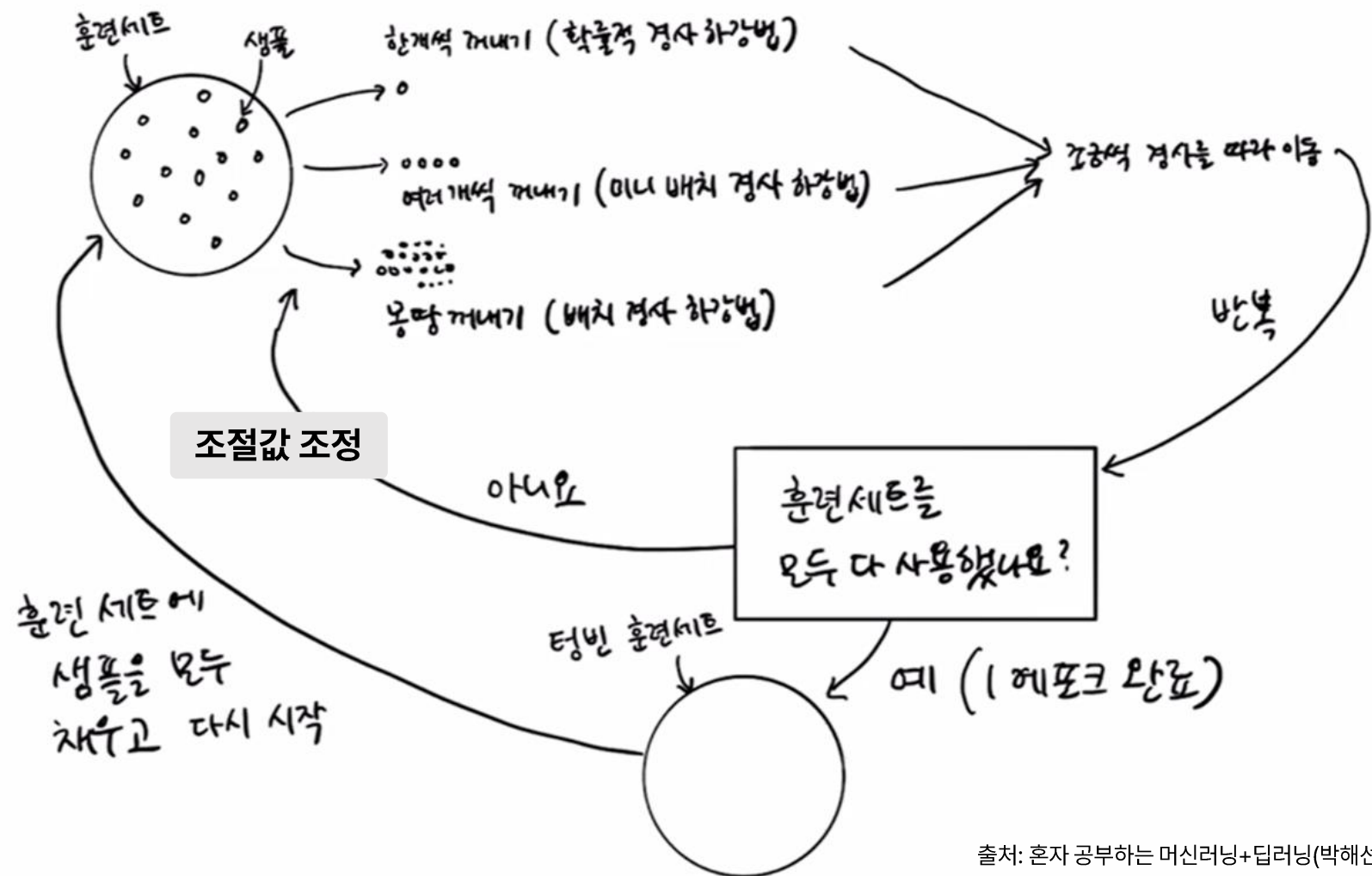
C: 훈련데이터의 학습강도. 높으면 많이 훈련



경사 하강법

Gradient Descent

학습 과정에서 예측값과 실제값의 차이(손실)를 최소화하기 위해 **조절값**을 반복적으로 조정하는 기법.
안개 낀 산에서 내려올 때 전체 길을 다 볼 수 없으니 발 주변만 보고 **가장 낮은 방향으로 한 걸음씩** 내려오는 것과 유사



- **배치:** 한 번에 꺼내는 샘플 데이터 덩어리
- **반복(Iteration):** 1개 배치로 1회 학습하는 동작
- **에포크(epoch):** 전체 훈련 데이터 세트를 한번 완전히 훈련하는 주기. 1 epoch은 N개의 반복으로 구성
- **손실함수:** 예측값과 실제값의 차이를 수치화한 함수. 이 손실을 최소화하는 방향으로 모델을 학습시킴

배치 경사하강법

각 배치에 전체 훈련 데이터 사용

느리지만 안정적

확률적 경사하강법(SGD) Stochastic Gradient Descent

각 배치에 샘플 1개씩 사용

빠르지만 불안정

미니배치 경사하강법

각 배치에 복수의 샘플 사용

속도와 안정성 균형

각 반복(Iteration)마다 조절값을 조금씩 조정하면서 학습이 조절값이 가중치이고 기술적으로는 모델 파라미터임

결정 트리(Decision Tree)

스무고개처럼 질문과 답을 반복하며 정답의 범위를 좁혀가는 학습 방법

01

불순도(Gini Impurity) 최소화

불순도는 다른 유형의 클래스가 섞여 있는 정도로 이 값을 최소화해야 함

02

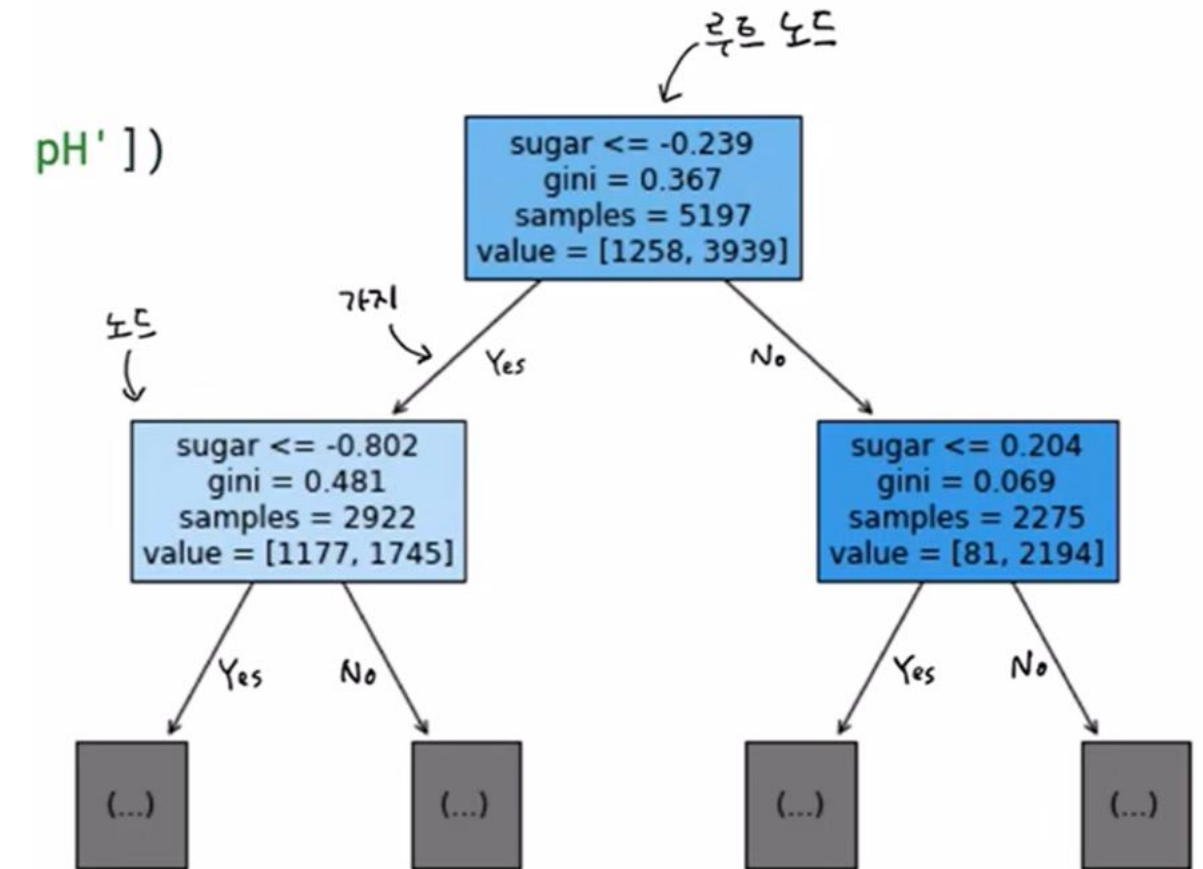
정보획득(Information Gain)을 최대화하는 질문을 선택

부모-자식 불순도 차이. 정보획득값이 크면 부모는 많이 섞여 있고 자식은 최소로 섞인 것

03

재귀적 분할

리프노드까지 반복



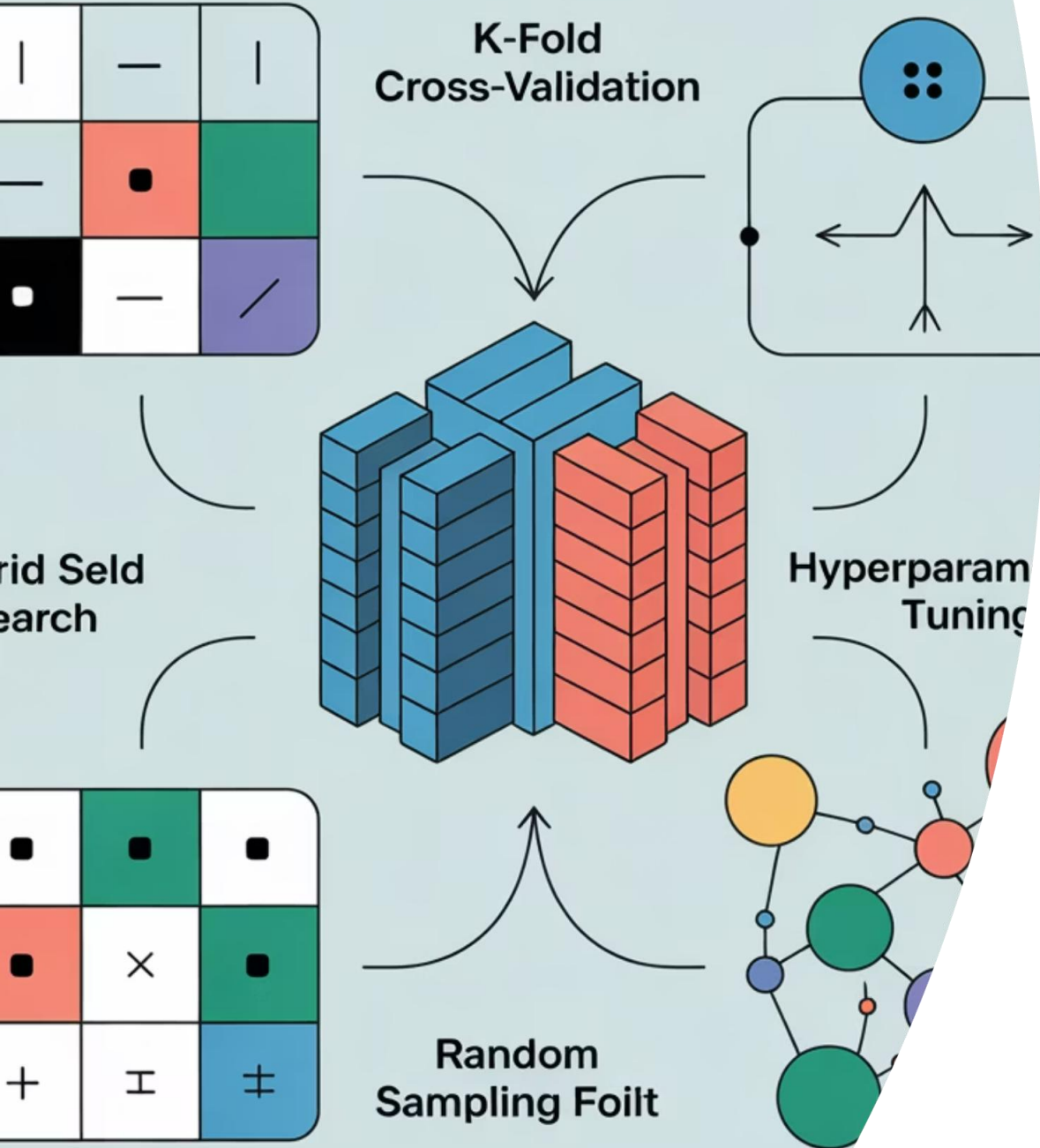
출처: 혼자 공부하는 머신러닝+딥러닝(박해선)

- gini: 불순도
- samples: 샘플 데이터 갯수
- values: 음성클래스 갯수와 양성클래스 개수
(갯수/총갯수가 0.5 초과면 양성 클래스)

장점: 데이터 전처리 불필요, 해석 용이

교차검증과 하이퍼파라미터 튜닝

Cross Validation



교차검증

[더 많은 설명](#)

데이터를 K개 폴드(데이터 그룹)로 분할
다양한 조합으로 훈련하여 성능 높이기

폴드 분할기:

- Kfold: 회귀모델
- StratifiedKFold(분류모델)



그리드 서치

[더 많은 설명](#)

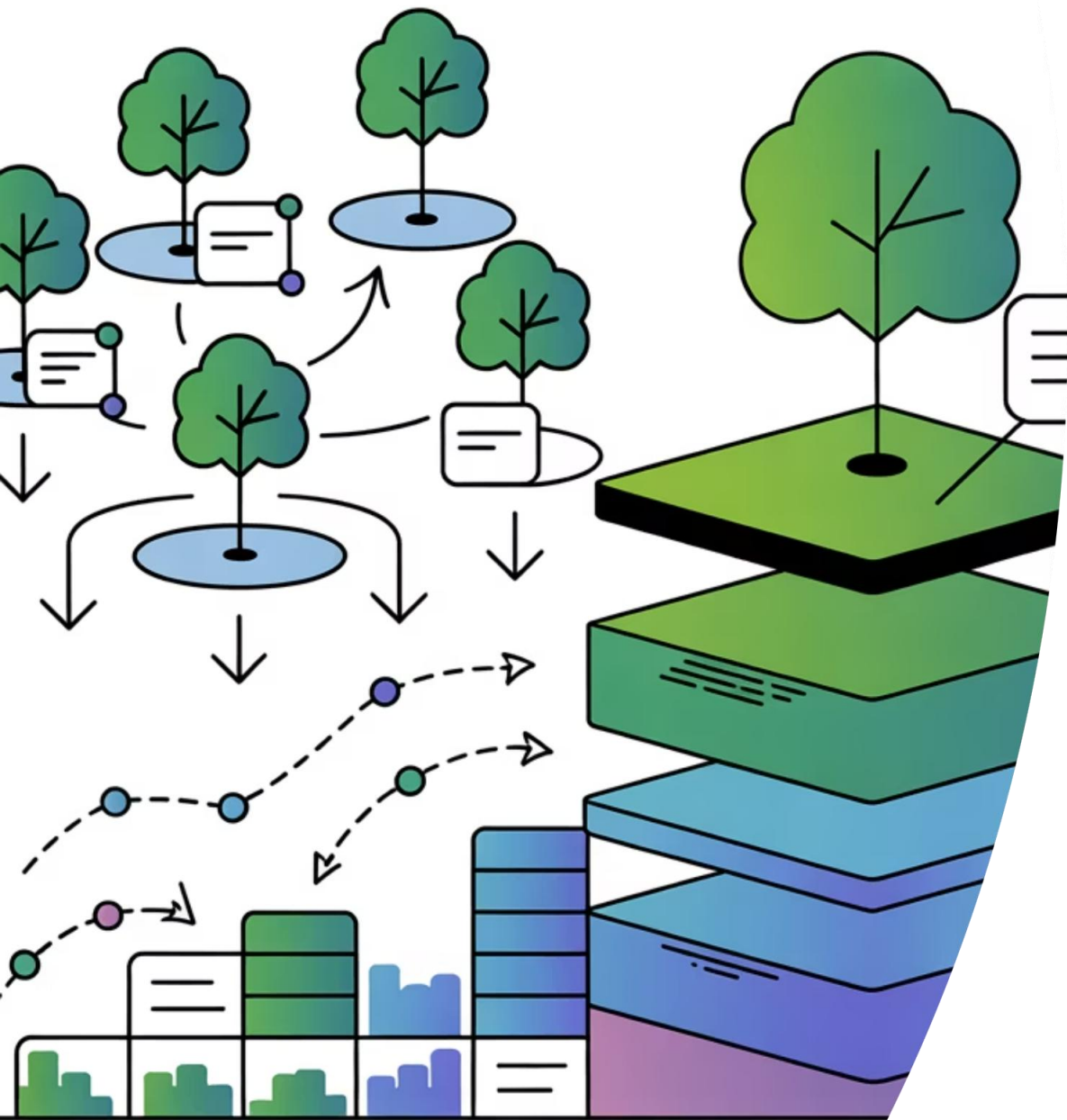
최적의 하이퍼 파라미터 탐색 목적
파라미터값을 변경하면서 훈련 및 검증
체계적이지만 느림



랜덤 서치

[더 많은 설명](#)

최적의 하이퍼 파라미터를 더 빠르게 찾기
파라미터값을 랜덤하게 뽑아 훈련 및 검증
충분한 탐색으로 좋은 결과를 낼 수 있음



앙상블 학습

Ensemble

여러 모델을 결합해 더 나은 성능을 얻는 알고리즘
배깅(Bagging), 부스팅(Boosting) 방식이 있음

랜덤포레스트

[더 많은 설명](#)

배깅 방식: 다수 모델의 예측 채택

1. 각 결정트리 모델별 랜덤 데이터 샘플링
2. 각 결정트리 모델별 랜덤 특성 선택
각 특성의 최적 분할점 자동 찾음
3. 각 결정트리 모델별 다수결 투표

엑스트라 트리

[더 많은 설명](#)

랜덤 포레스트보다 더 빠르면서 과대적합 줄임

1. 각 결정트리 모델이 100% 훈련세트로 학습
2. 각 결정트리별 랜덤 특성 선택.
각 특성의 분할점은 랜덤하게 선택
3. 각 결정트리별 다수결 투표

그레이디언트 부스팅

[더 많은 설명](#)

부스팅 방식: 모델 순차적 학습. 점진적 모델 개선

Learning rate(가중치 조정 비율)만큼 이전 실수를 보완하면서 학습

히스토그램 GB

[더 많은 설명](#)

데이터 구간화로 속도 향상(10~100배)

메모리 효율적

대용량 데이터 처리

XGBoost와 LightGBM이 실무에서 가장 많이 사용됨. LightGBM은 대용량 학습에 사용.

비지도학습: K-Mean 알고리즘

01

랜덤 중심점(센트로이드: centroid) 선정

02

가까운 데이터 라벨링

03

중심점 이동

04

라벨링과 중심점 이동을 반복하면서 더 이상 움직일 필요 없을때까지 반복

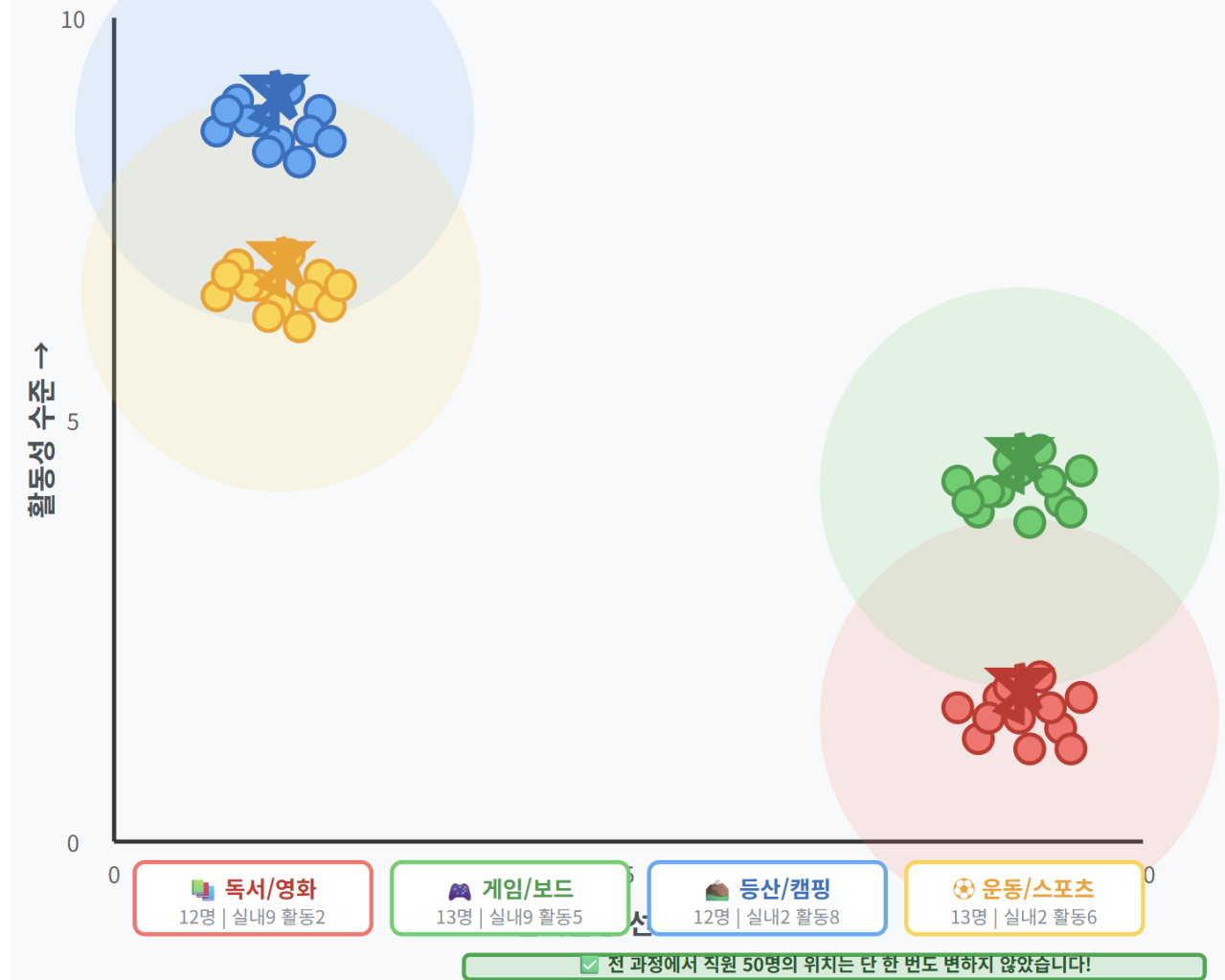
최초 K값(클러스터 갯수)에 따라 결과가 달라질 수 있음
몇 개의 클러스터가 있을 지 알 수 없는데 미리 지정해야 하기 때문임

엘보우 방법으로 최적 K값 찾기

- K값을 늘리면서 클러스터의 응집도(inertia)의 추이를 봄
- 팔꿈치 처럼 꺾이는 지점의 K값이 최적

동호회 나누기 4단계: 라벨 부여 → 중심점 이동을 반복 → 완성! 🎉

더 이상 변화 없음 → 취미별 동호회 자동 분류 완료!



[동작원리 예시](#)

PCA: 주성분 분석

Principle Component Analysis

중요한 특징값은 최대한 유지하면서, 덜 중요한 특징값은 버림으로써 데이터를 가장 잘 설명하는 축(주성분)을 찾는 것

벡터

특성 값의 배열을 의미
예) 키, 몸무게 벡터: [171, 80]

차원 축소

- 선/면/입체 차원 축소 아님
- 벡터 값 갯수를 줄이는 것
- 벡터값에서 중요한 것만 선택하는 방식(Feature selection)과 새로운 벡터값으로 차원을 축소하는 방법(Feature transformation)이 있음
- 대표적인 방식이 주성분 분석(PCA:Principle Component Analysis)임

1

데이터 중심화

특성 점수 - 특성 평균하여
0점 중심 데이터 재배치

2

공분산 행렬

특성 간 상관 관계

3

고유값과 고유벡터 계산

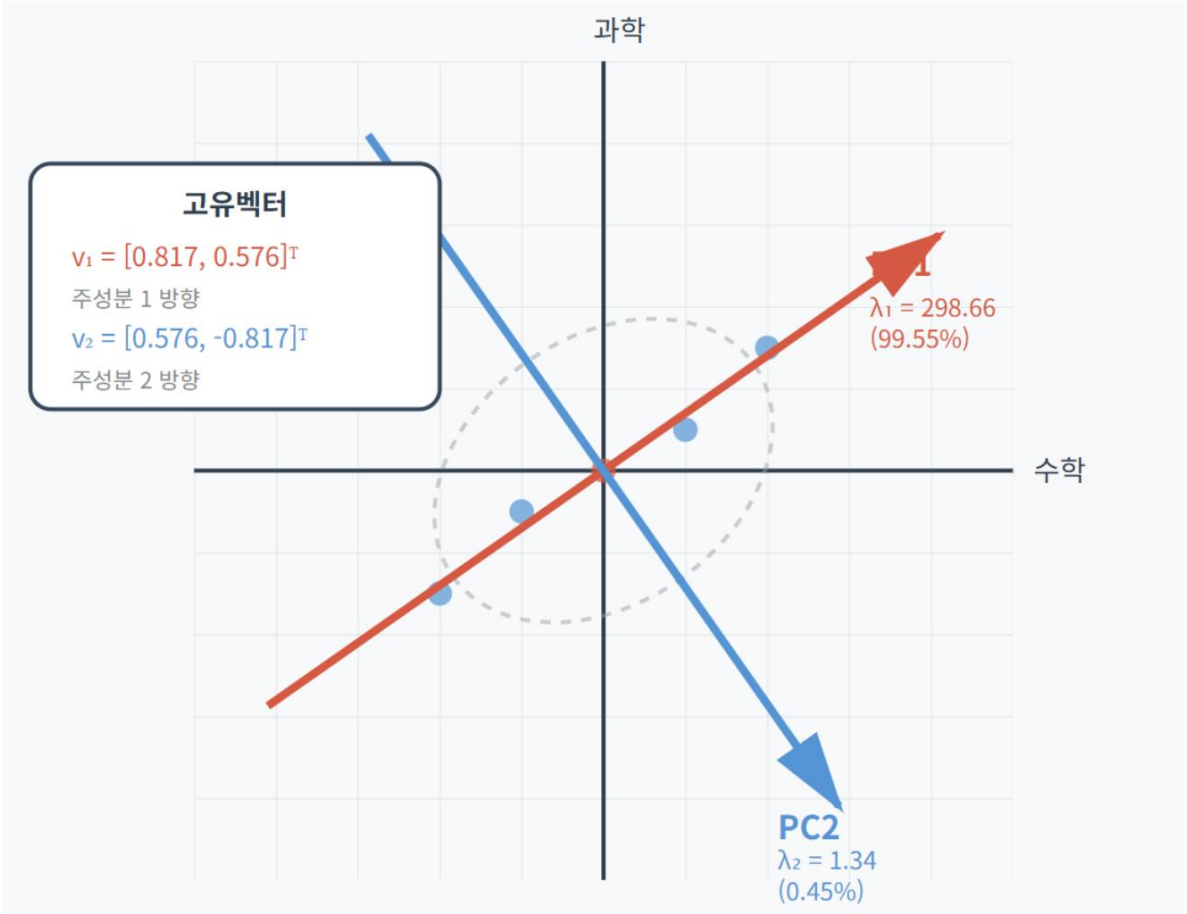
고유벡터는 퍼진 방향이고 고유값은
그 방향에 있는 데이터량

4

차원 축소

제 1 주성분으로 차원 축소된 벡터 계산

[동작원리 예시](#)



주성분=데이터가 가장 크게 퍼진 방향

PC1(주성분1)은 전체 분산의 99.55%를 설명.

즉 이 방향으로만 투영해도 정보 손실이 거의 없음

수학과 과학의 2차원 벡터를 종합학업능력이라는 1차원 벡터로 차원 축소

딥러닝

여러 겹의 필터로 깨끗한 물을 만드는 정수기처럼 **여러 층의 신경망**을 통해 **데이터에서 스스로 패턴**을 찾아내는 **머신러닝** 방법
각 신경망 층 내의 뉴런(=노드=유닛)이 정보를 받아 중요도에 따라 가중치를 곱해 다음에 전달함으로써 패턴을 찾아냄

인공신경망(ANN)

[예시 보기](#)

입력층-은닉층-출력층으로 구성. 각 층에 복수 뉴런(노드)
활성화 함수: 은닉층과 출력층 결과값을 비선형으로 변환
순전파로 예측하고 역전파로 가중치 조정하며 오차 최소화

심층신경망(DNN: Deep Neural Network)

3개 이상 은닉층을 만들어 복잡한 패턴 학습

주요 용어:

- Flatten층: 1차원 배열로 변환한 입력 층
- 훈련률: 한 번에 조정할 가중치
- momentum: 이전 방향성 유지율(0~1)
- 옵티마이저: 훈련률 자동 조정 알고리즘. Adam을 많이 사용

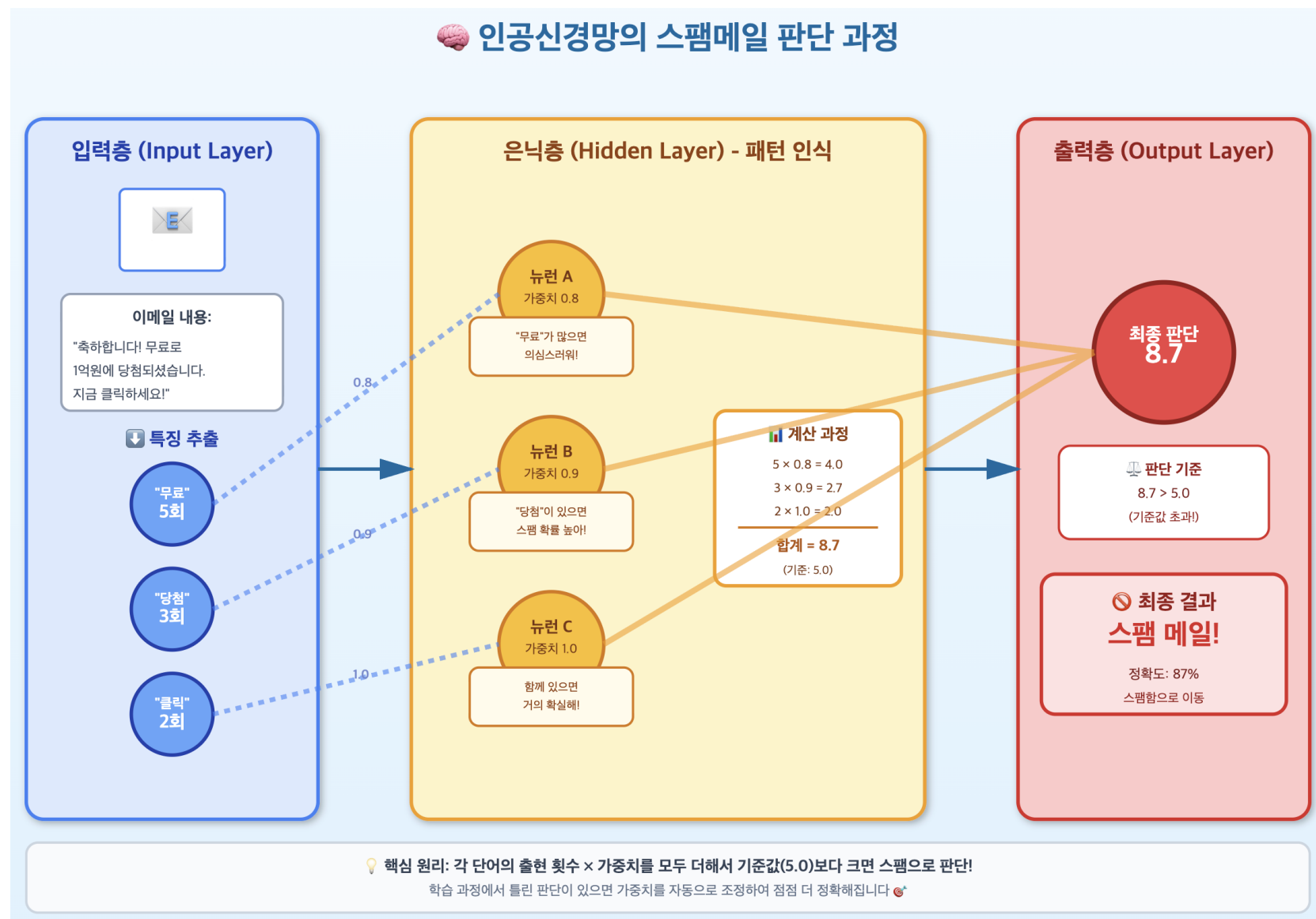
특화 구조

합성곱신경망(CNN): 이미지/영상 데이터 모델링

순환신경망(RNN): 순서 있는 텍스트 모델링

트랜스포머(Transformer): 대규모 언어 모델링

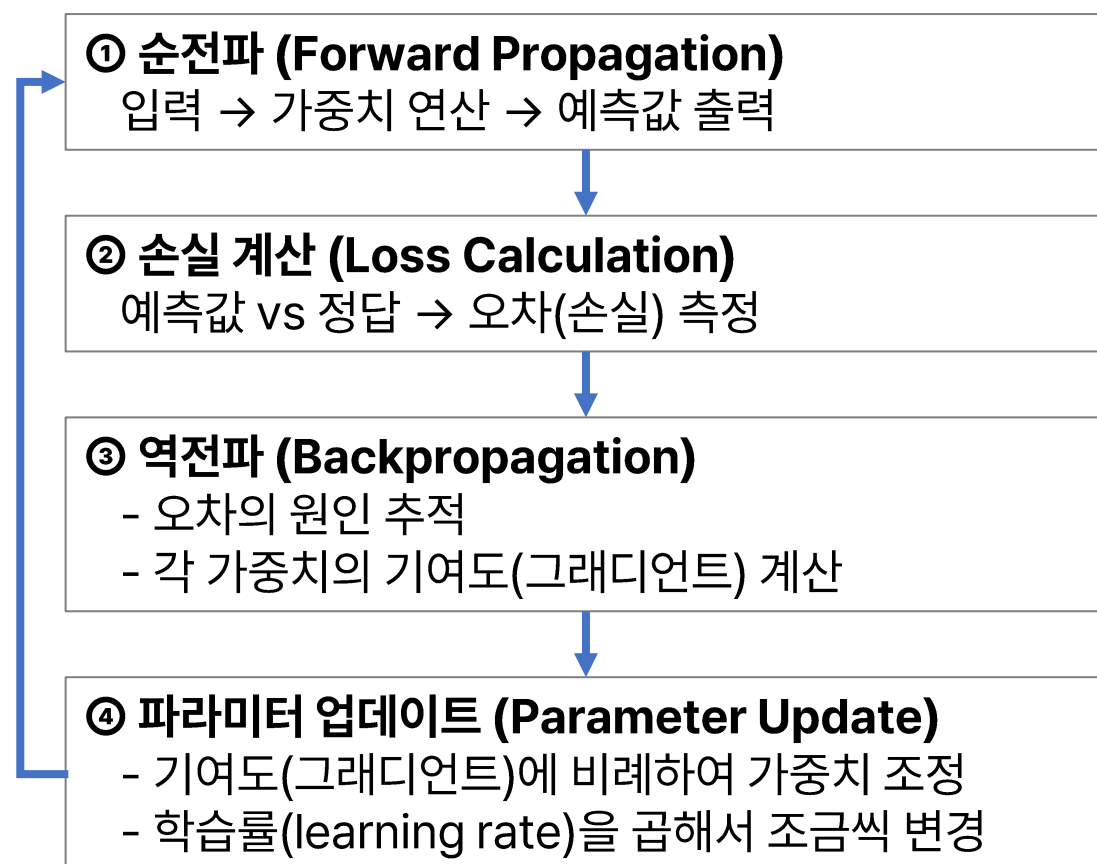
딥러닝 개발 프레임워크: 파이토치, 텐서플로우, 케라스, 잭스



딥러닝

인공신경망의 학습 원리

"순전파 → 역전파 → 파라미터 업데이트"를 반복하면서
최고의 정답을 예측하는 모델을 만드는 것



양궁 선수 비유

① 순전파 = 화살 쏘기

선수 🏹 → (슈우욱~) → 🎯 "땅!" → 7점

② 손실 계산 = 얼마나 빔나갔나?

목표: 10점 (정중앙)

실제: 7점

차이: 3점 ✖

③ 역전파 = 왜 빔나갔나?

원인 분석:



④ 업데이트 = 자세 교정

팔 각도: 45° → 45.04° (조금만 수정)

활 장력: 50 → 50.03 (조금만 수정)

🔄 반복

1회: 7점 → 자세 수정

2회: 8점 → 자세 수정

3회: 9점 → 자세 수정

...

100회: 9.8점 → 거의 완벽!

신경망 훈련 모범사례

검증 손실(validation loss) 비교: 과대/과소 적합 파악

훈련 시 검증 데이터 세트로 각 Epoch(학습주기)별 손실값 추이 파악

검증 데이터 손실이 훈련 데이터 손실보다 크면 과대적합임

드롭아웃(Dropout)으로 과적합 줄이기

각 Epoch에 모든 뉴런 사용하면 특정 뉴런에 과도하게 의존하여 과적합 가능

훈련 시 각 Epoch의 뉴런 중 무작위로 일정 비율은 비활성화: 20%~50%

모델 저장과 복원: 기존 모델 위에 추가 튜닝하여 모델 생성

모델 층 구조, 훈련 환경 설정(손실함수, 측정 지표 등), 가중치 등 저장 및 복원

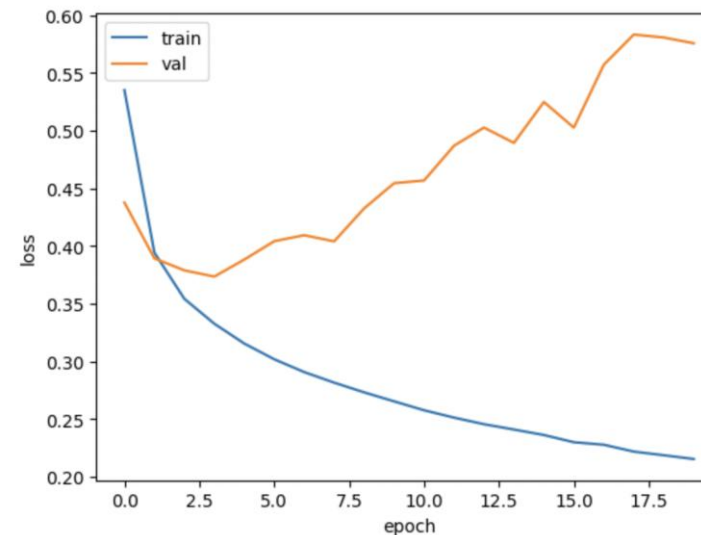
가중치만 저장 및 복원도 가능

콜백: 훈련 과정 중 최고 성능 모델을 자동 저장

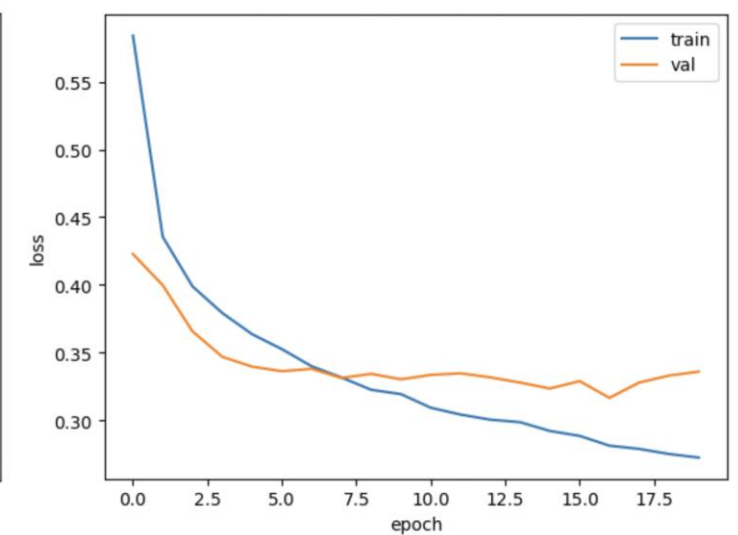
검증손실이 이전 최소값보다 낮아지면 해당 모델을 자동 저장

조기종료(Early Stopping): 검증 손실이 일정 에포크 동안 미 개선 시 중단

검증 손실 추이



드롭아웃 결과



```
# 최고 성능 모델 저장
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    'best-model.keras',
    save_best_only=True,
    monitor='val_loss',
    verbose=1
)

# 조기 종료
early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=5,           # 몇개까지의 개선 없는 에포크 수를 기다릴것인가?
    monitor='val_loss',   # 모니터링 대상 (기본값)
    restore_best_weights=True, # 최고 성능 가중치로 복원
    verbose=1            # 중단 시 메시지 출력
)

# 두 콜백을 함께 사용
model.fit(train_scaled, train_target,
          epochs=100, # 충분히 큰 값 설정
          validation_data=(val_scaled, val_target), # 필수!
          callbacks=[checkpoint_cb, early_stopping_cb])
```

합성곱 신경망 (CNN)

Convolutional Neural Network

이미지나 영상 데이터 모델링에 사용되며 파라미터 수를 최소로 줄여 가성비 있게 최고 성능을 내는 것이 목적

지역연결과 가중치 공유

[예시 보기](#)

파라미터를 줄이는 방법

- 지역연결: 일정 크기 영역으로 나누어 계산
- 가중치공유: 각 영역은 동일한 가중치 사용

주요 용어

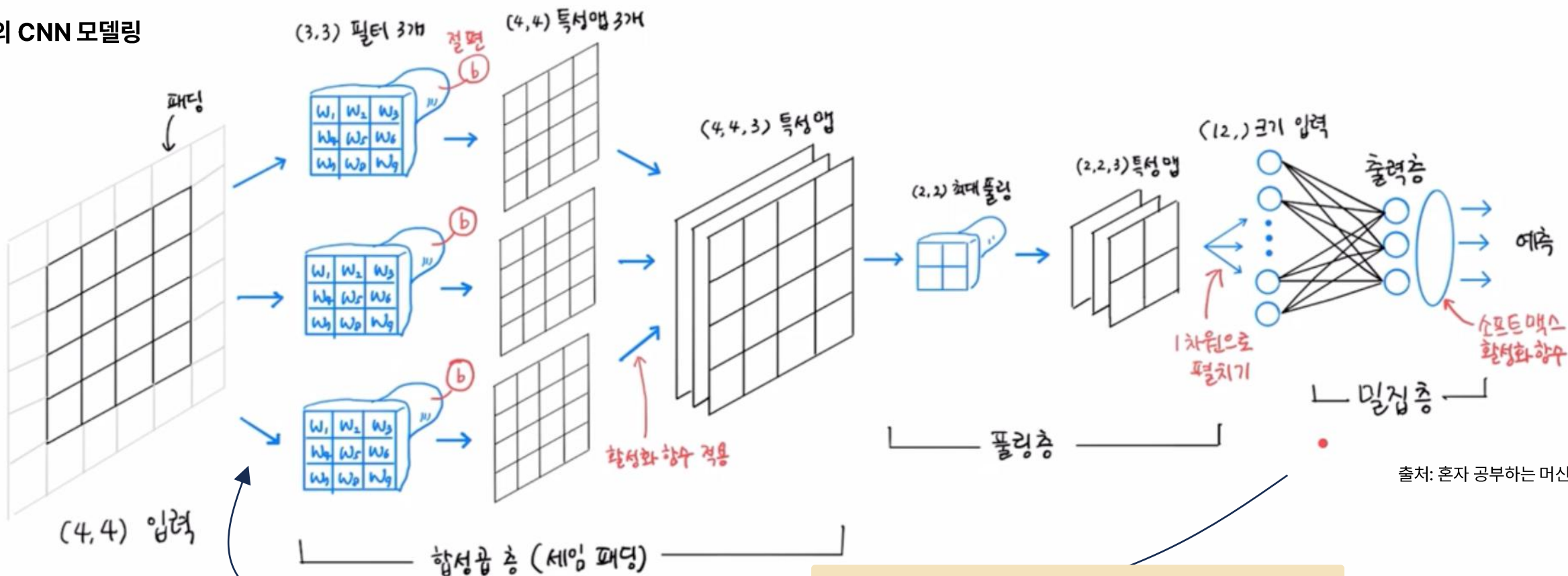
[예시 보기](#)

- 채널: 정보가 흐르는 층. 흑백은 1, RGB는 3개
- 커널: 특징을 찾는 일정 크기 창. 가중치 소유
- 필터: 특정 특징 추출기. 채널별 가중치 상이
- 특성맵: 커널이 이동하면서 출력하는 값 행렬

패딩과 풀링

- 패딩: 출력크기 유지를 위해 가장자리에 추가 픽셀을 덧붙이는 기법 (보통 Zero패딩)
- 풀링: 메모리 절감과 학습속도 향상을 위해 특성맵의 차원을 줄이는 것

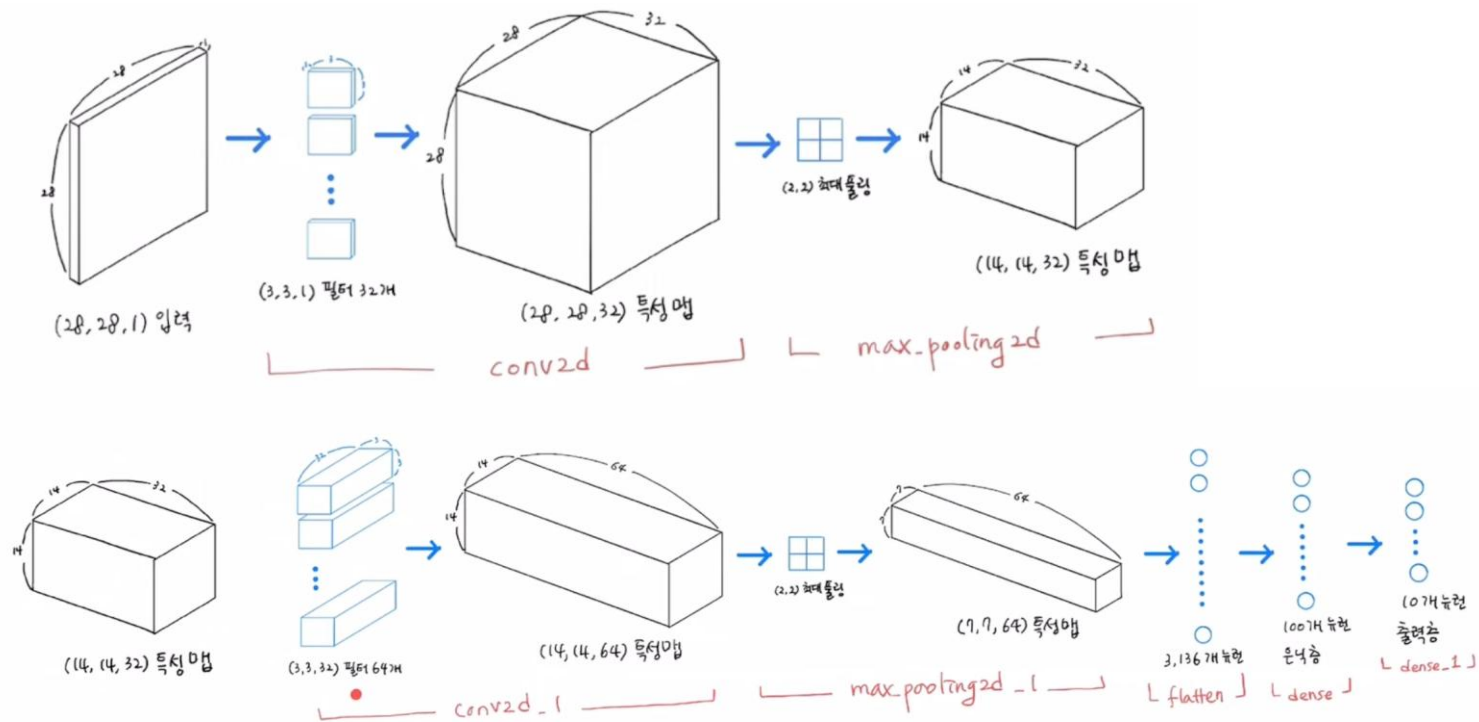
흑백 이미지의 CNN 모델링



출처: 혼자 공부하는 머신러닝+딥러닝(박해선)

손실계산 -> 역전파 -> 파라미터(가중치) 업데이트

합성곱 신경망 (CNN)



출처: 혼자 공부하는 머신러닝+딥러닝(박해선)

3. 모델 환경설정(compile)과 훈련(fit)

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

```
checkpoint_cb = keras.callbacks.ModelCheckpoint('best-cnn-model.keras',  
                                              save_best_only=True)
```

```
early_stopping_cb = keras.callbacks.EarlyStopping(patience=2,  
                                                  restore_best_weights=True)
```

```
history = model.fit(train_scaled, train_target, epochs=20,  
                   validation_data=(val_scaled, val_target),  
                   callbacks=[checkpoint_cb, early_stopping_cb])
```

1. 첫번째 합성곱 층 구성

```
model = keras.Sequential()
```

```
model.add(keras.layers.Conv2D(32, kernel_size=3, activation='relu',  
                              padding='same', input_shape=(28,28,1)))
```

```
model.add(keras.layers.MaxPooling2D(2))
```

2. 두번째 합성곱 층 구성

```
model.add(keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu',  
                              padding='same'))
```

```
model.add(keras.layers.MaxPooling2D(2))
```

```
model.add(keras.layers.Flatten())
```

```
model.add(keras.layers.Dense(100, activation='relu'))
```

```
model.add(keras.layers.Dropout(0.4))
```

```
model.add(keras.layers.Dense(10, activation='softmax'))
```

4. 예측

```
preds = model.predict(val_scaled[0:1])  
print(preds)
```

결과: 9번째 Class로 예측함

```
classes = ['티셔츠', '바지', '스웨터', '드레스', '코트',  
          '샌달', '셔츠', '스니커즈', '가방', '앵클 부츠']
```

```
[[2.4567913e-17 4.5715461e-23 8.8957614e-20 6.0725004e-18 8.0334706e-18  
 1.5484204e-18 1.5557679e-18 6.1831409e-16 1.0000000e+00 3.3490770e-19]]
```

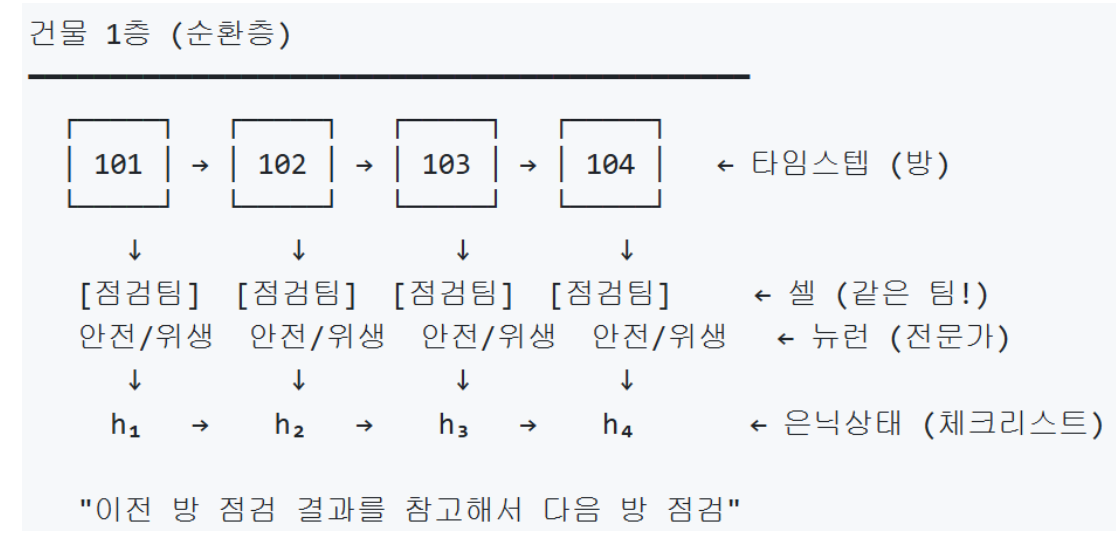
순환 신경망 (RNN)

Recurrent Neural Network

순서가 있는 데이터에서 이전 정보를 기억하며 다음에 올 것을 예측하는 인공지능 신경망

주요용어

- 순환층: RNN이 작동하는 한 겹의 층
- 셀: 매번 같은 방식으로 입력을 처리하는 계산기
- 타임스텝: 입력을 처리하는 각 단계
- 은닉상태: 각 타임스텝의 결과로 누적된 기억



뉴런간 상호참조

현재 시점 각 영역은 이전의 모든 영역을 참조:

- 언어영역(r₁): 이전 언어+문법+감정 → "장소 추가"
- 문법영역(r₂): 이전 언어+문법+감정 → "목적어 확인"
- 감정영역(r₃): 이전 언어+문법+감정 → "학교=일상"

→ 뇌의 각 영역이 다른 영역의 정보도 통합하여 처리

→ 이것이 RNN에서 뉴런 간 상호 참조의 의미

RNN 구조: "I am a boy" 처리 과정

순환층=1, 셀=1, 타임스텝=4, 은닉상태=4, 뉴런=3



은닉 상태 업데이트 (Hidden State Update)

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

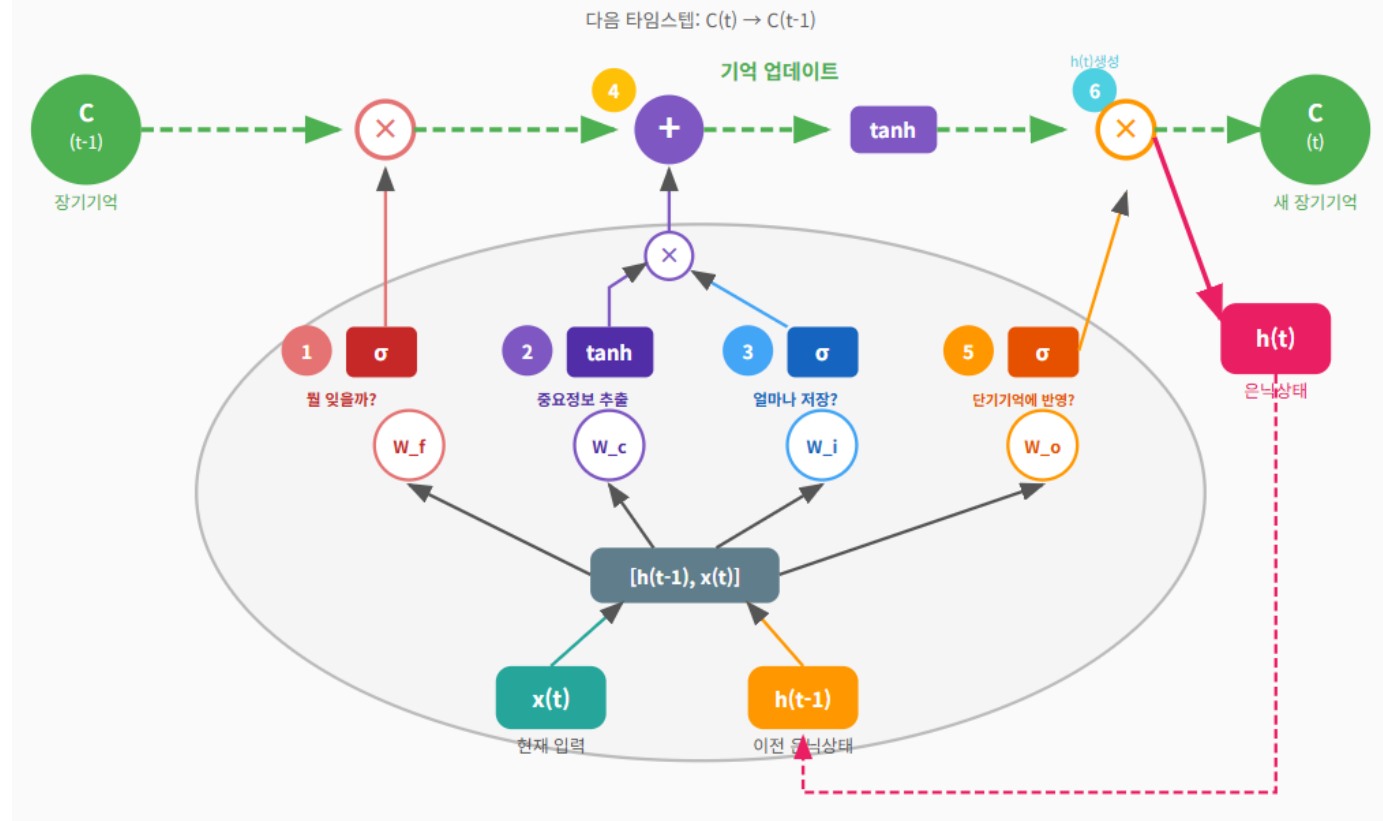
- 용어 정리
- 순환층 = 파란 점선 영역 (1개)
- 셀 = 녹색 박스 (1개, 재사용)
- 타임스텝 = t=1~4 (입력 단어 수)
- 문법 뉴런 → h[0]
- 시제 뉴런 → h[1]
- 감정 뉴런 → h[2]
- 은닉상태 h = [문법, 시제, 감정] (3차원)

- 각 타임스텝은 동일한 가중치(W_h, W_x) 사용
- 활성화 함수 tanh는 -1과 1 사이로 출력값을 스케일링

LSTM/GRU 셀

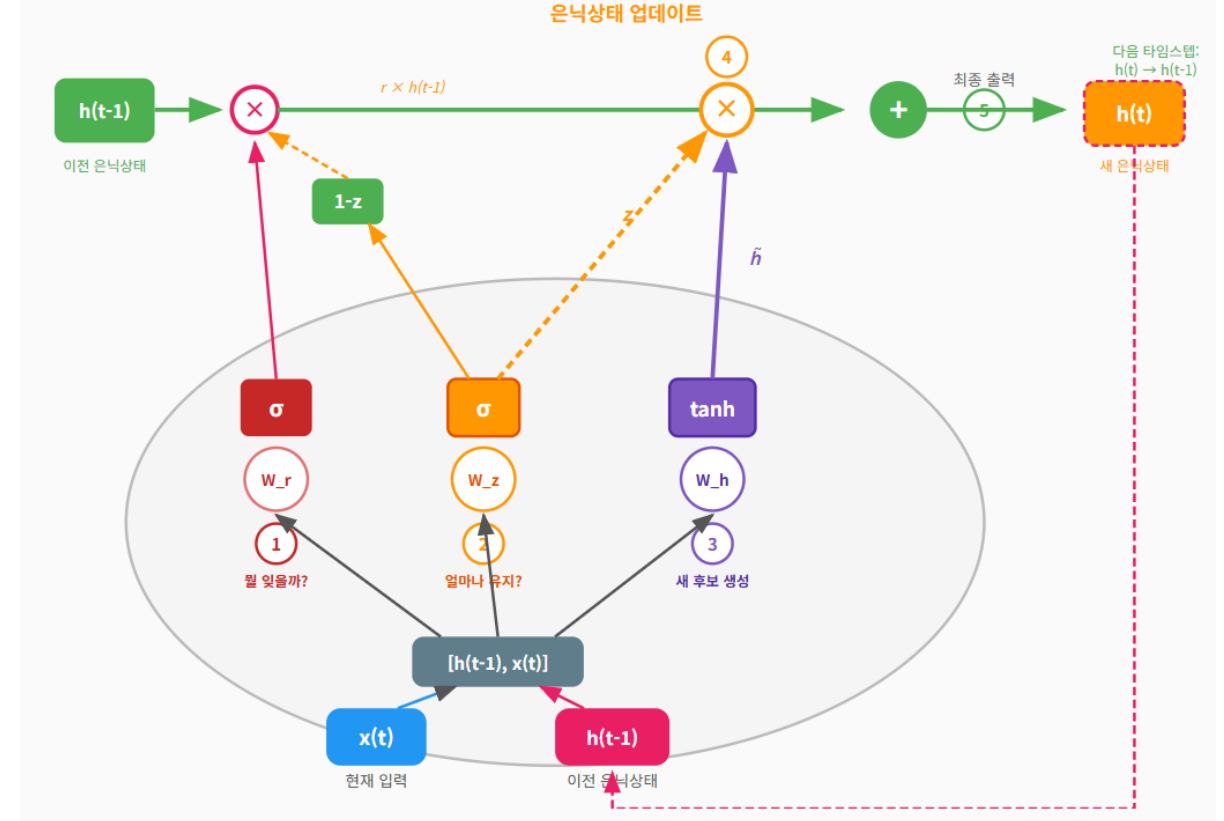
LSTM(Long Short-Term Memory)셀은 오래된 정보도 필요하면 기억하고, 불필요한 건 잊는 똑똑한 기억 시스템을 만들기 위한 RNN 셀이고, GRU(Gated Recurrent Unit)셀은 LSTM셀을 단순화하여 좀 더 빠르게 학습할 수 있는 RNN셀임

LSTM 셀



예시 보기

GRU 셀



- 망각 게이트: 이전 기억 중 불필요한 부분을 선택적으로 삭제
- 입력 게이트: 새 정보를 얼마나 저장할지 결정
- 셀 게이트: 현재 입력에서 중요한 정보 추출
- 출력 게이트: 장기 기억(C)에서 얼마나 읽어와서 새 단기기억(은닉상태)에 반영할 지 결정

* 단기기억인 은닉상태가 셀의 최종 결과

- 리셋 게이트: 이전 기억을 얼마나 무시할지 결정
- 업데이트 게이트: 새 정보와 기존 정보의 혼합 비율 결정
- 후보 생성: 리셋된 이전 기억과 현재 입력으로 새 기억 후보 생성

LSTM/GRU 셀

100개의 리뷰 각각에 대해 긍정/부정 확률을 판단하는 모델 학습 예시

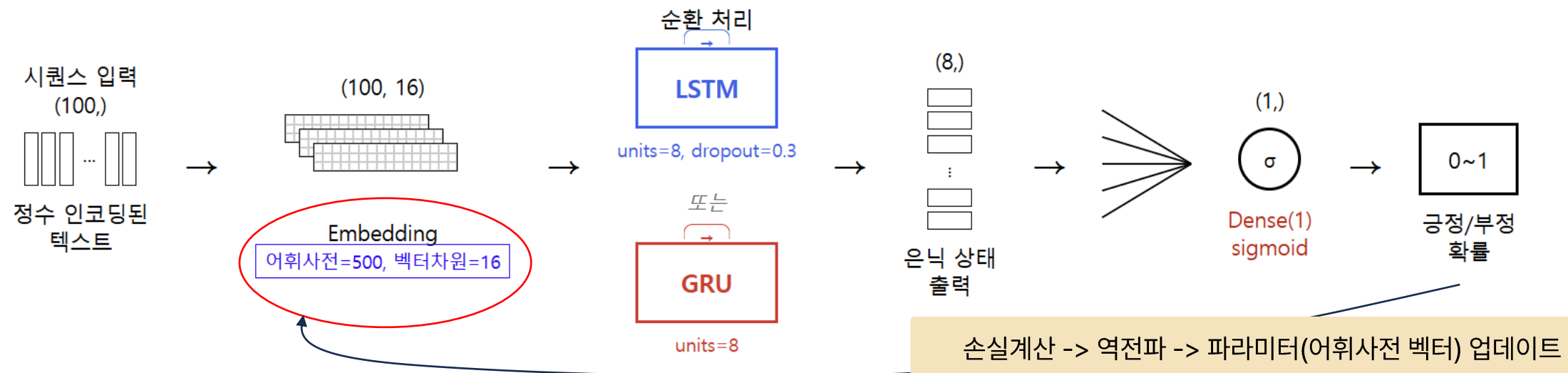
```
model = keras.Sequential()
model.add(keras.layers.Embedding(500, 16, input_shape=(100,)))
model.add(keras.layers.LSTM(8, dropout=0.3))
model.add(keras.layers.Dense(1, activation='sigmoid'))
rmsprop = keras.optimizers.RMSprop(learning_rate=1e-4)

model.compile(optimizer=rmsprop, loss='binary_crossentropy',
              metrics=['accuracy'])
checkpoint_cb = keras.callbacks.ModelCheckpoint('best-dropout-model.keras',
                                              save_best_only=True)
early_stopping_cb = keras.callbacks.EarlyStopping(patience=3,
                                                  restore_best_weights=True)
history = model.fit(train_seq, train_target, epochs=100, batch_size=64,
                  validation_data=(val_seq, val_target),
                  callbacks=[checkpoint_cb, early_stopping_cb])
```

```
model = keras.Sequential()
model.add(keras.layers.Embedding(500, 16, input_shape=(100,)))
model.add(keras.layers.GRU(8))
model.add(keras.layers.Dense(1, activation='sigmoid'))
rmsprop = keras.optimizers.RMSprop(learning_rate=1e-4)

model.compile(optimizer=rmsprop, loss='binary_crossentropy',
              metrics=['accuracy'])
checkpoint_cb = keras.callbacks.ModelCheckpoint('best-gru-model.keras',
                                              save_best_only=True)
early_stopping_cb = keras.callbacks.EarlyStopping(patience=3,
                                                  restore_best_weights=True)
history = model.fit(train_seq, train_target, epochs=100, batch_size=64,
                  validation_data=(val_seq, val_target),
                  callbacks=[checkpoint_cb, early_stopping_cb])
```

- 토큰(Token): 텍스트를 의미 있는 최소 단위(예: 단어)로 분리한 것
- 어휘사전(Vocabulary): 말뭉치(Corpus: 특정 목적을 위해 수집된 대량의 텍스트 데이터 집합)에 등장하는 고유 토큰들을 숫자 인덱스로 매핑한 목록 (예: He->10, loves->14)



임베딩(Embedding)

단어를 컴퓨터가 이해할 수 있는 숫자들의 조합으로 변환하는 기술이며 학습을 통해 비슷한 의미를 가진 단어끼리 비슷한 숫자 패턴을 갖게 됨

어휘사전 구축

어휘 사전 (Vocabulary):

ID	단어
0	<PAD>
1	I
2	am
3	a
4	boy
5	girl
6	you
7	are
8	the
9	happy

패딩용

문장을 정수 인덱스로 변환

입력 문장: "I am a boy"

토큰화:

["I", "am", "a", "boy"]

정수 인덱스로 변환:

[1, 2, 3, 4]

어휘사전 임베딩(=임베딩행렬=임베딩 테이블) 생성

임베딩 테이블: 주어정도, 성별관련도, 생명체 정도, 나이수준, 추상성 정도

ID	5차원 벡터	
0	[0.00, 0.00, 0.00, 0.00, 0.00]	# <PAD>
1	[0.23, -0.45, 0.67, 0.12, -0.34]	# I
2	[-0.12, 0.56, 0.23, -0.67, 0.45]	# am
3	[0.45, 0.12, -0.23, 0.78, -0.56]	# a
4	[-0.34, 0.78, 0.45, -0.12, 0.67]	# boy
5	[-0.32, 0.75, 0.43, -0.15, 0.69]	# girl
6	[0.56, -0.23, 0.34, 0.45, -0.12]	# you
7	[-0.15, 0.58, 0.25, -0.65, 0.42]	# are
8	[0.42, 0.15, -0.25, 0.75, -0.53]	# the
9	[0.67, 0.89, -0.45, 0.23, 0.12]	# happy

임베딩 벡터

단어	5차원 임베딩 벡터
I	[0.23, -0.45, 0.67, 0.12, -0.34]
am	[-0.12, 0.56, 0.23, -0.67, 0.45]
a	[0.45, 0.12, -0.23, 0.78, -0.56]
boy	[-0.34, 0.78, 0.45, -0.12, 0.67]

학습을 반복 하면서
어휘사전 벡터값을
비슷한 의미의 단어
는 유사한 숫자 패턴
을 가지도록 변경

주어(subject) 차원이 높아짐

I → [0.85, -0.12, 0.34, 0.15, -0.23] # 1인칭

you → [0.82, -0.15, 0.31, 0.18, -0.21] # 2인칭

첫 번째 값(주어 특성)이 비슷!

성별 차원

boy → [-0.34, 0.78, 0.45, -0.12, 0.67] # 남성

girl → [-0.32, 0.75, 0.43, -0.15, 0.69] # 여성

벡터가 매우 유사!

인코더-디코더 순환 신경망

한 형태의 정보를 다른 형태로 변환(번역, 요약 등)하기 위한 순환신경망

인코더(Encoder): 이해하는 단계

입력 문장을 순차적으로 읽음

전체 의미를 하나의 컨텍스트 벡터(고정 길이 숫자 배열)로 요약

마치 책 한 권을 읽고 핵심 요약문을 만드는 것

디코더(Decoder): 이해한 것을 표현하는 단계

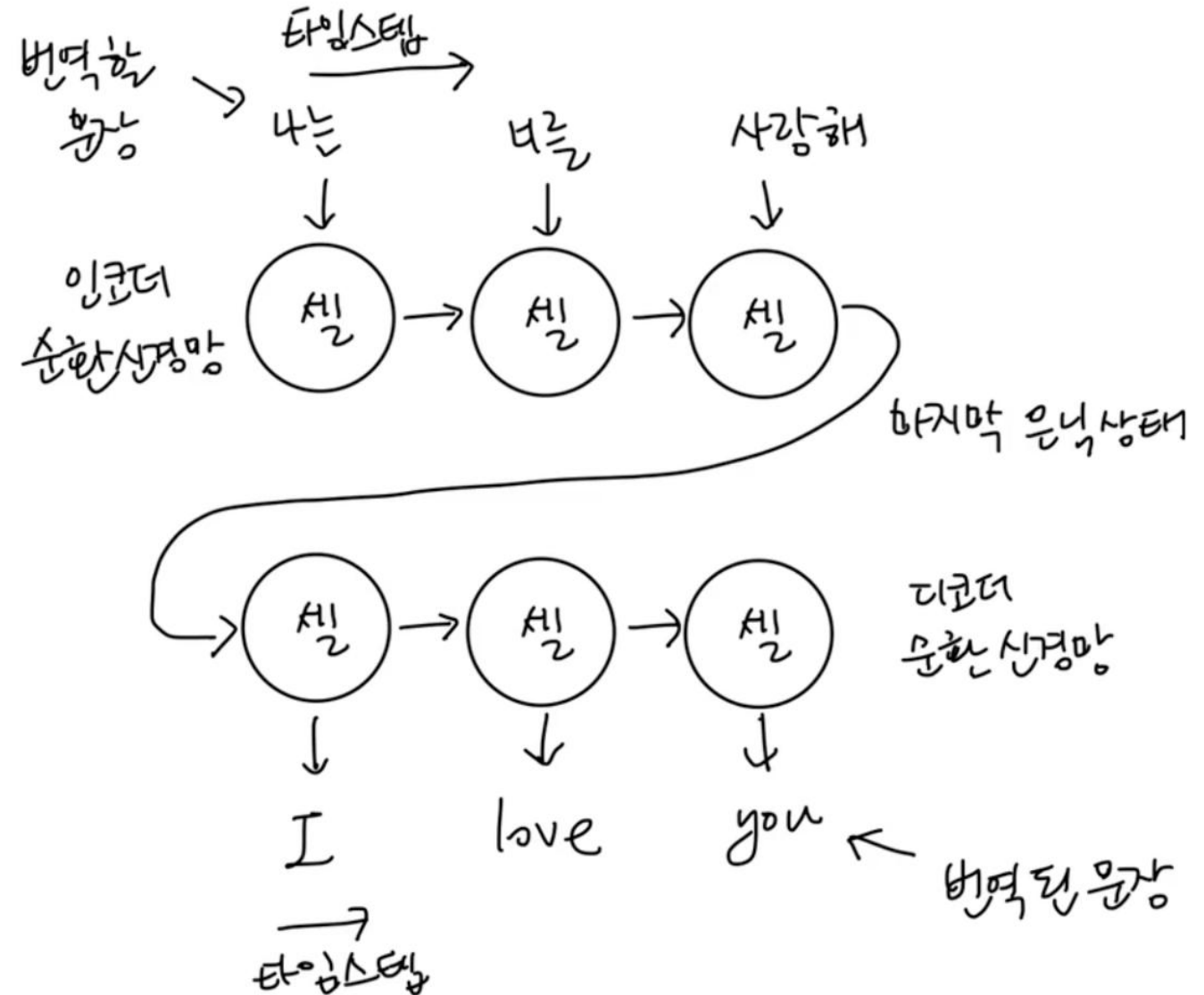
컨텍스트 벡터를 받아서 한 단어씩 순차적으로 출력을 생성

마치 요약문을 보고 새로운 글을 쓰는 것

문제점: 정보 손실 발생

아무리 긴 문장이라도 고정된 크기의 벡터 하나에 담아야

하기 때문에 정보 손실 발생 가능성 있음

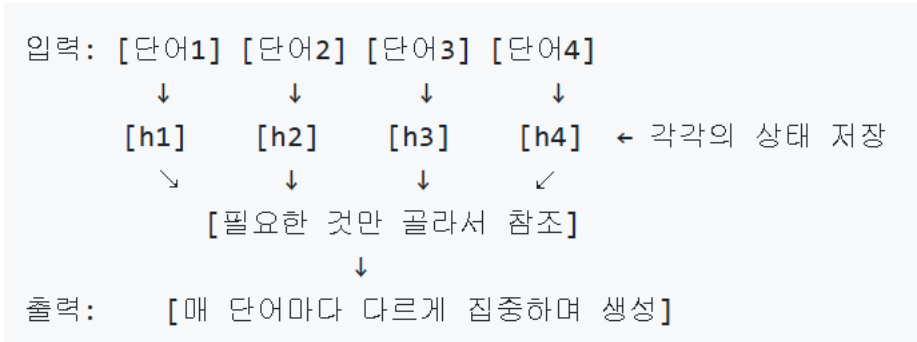


출처: 혼자 공부하는 머신러닝+딥러닝(박해선)

어텐션 메커니즘

전체 내용 중 중요한 부분에 집중하기 위한 메커니즘

인코더 각 토큰의 은닉상태를 디코더에서 참조

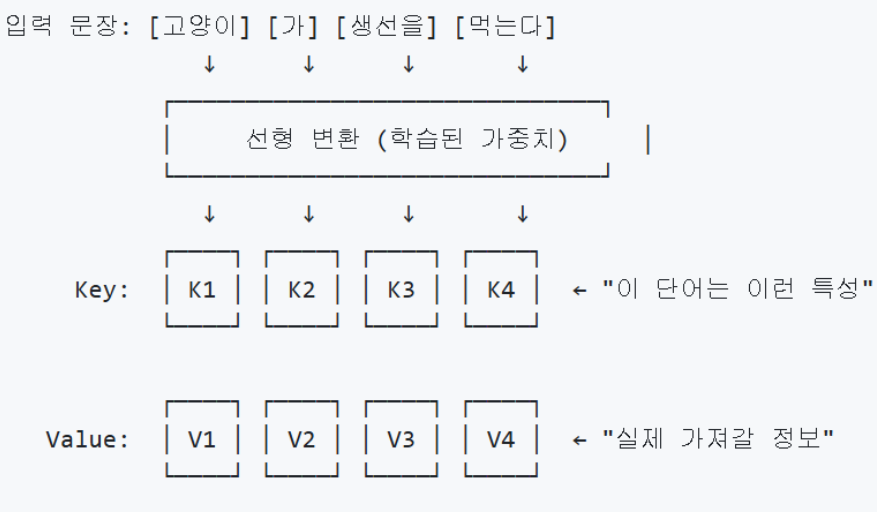


각 토큰의 중요도를 계산해 중요한 부분에 집중

- Query 가중치 행렬: 상황별 질문 가중치 행렬
- Key 가중치 행렬: 각 토큰의 특징 가중치 행렬
- Value 가중치 행렬: 각 토큰의 실제 정보 행렬

문제점

- 순차적 처리: t번째 단어 처리 위해 t-1번째까지 완료 필요
- 학습 속도 저하: 긴 문장일수록 학습 시간이 선형적으로 증가
- 장거리 의존성 문제: 문장이 길수록 앞쪽 정보 희석됨



Q/K/V 예시

Query (질문자) 관점:

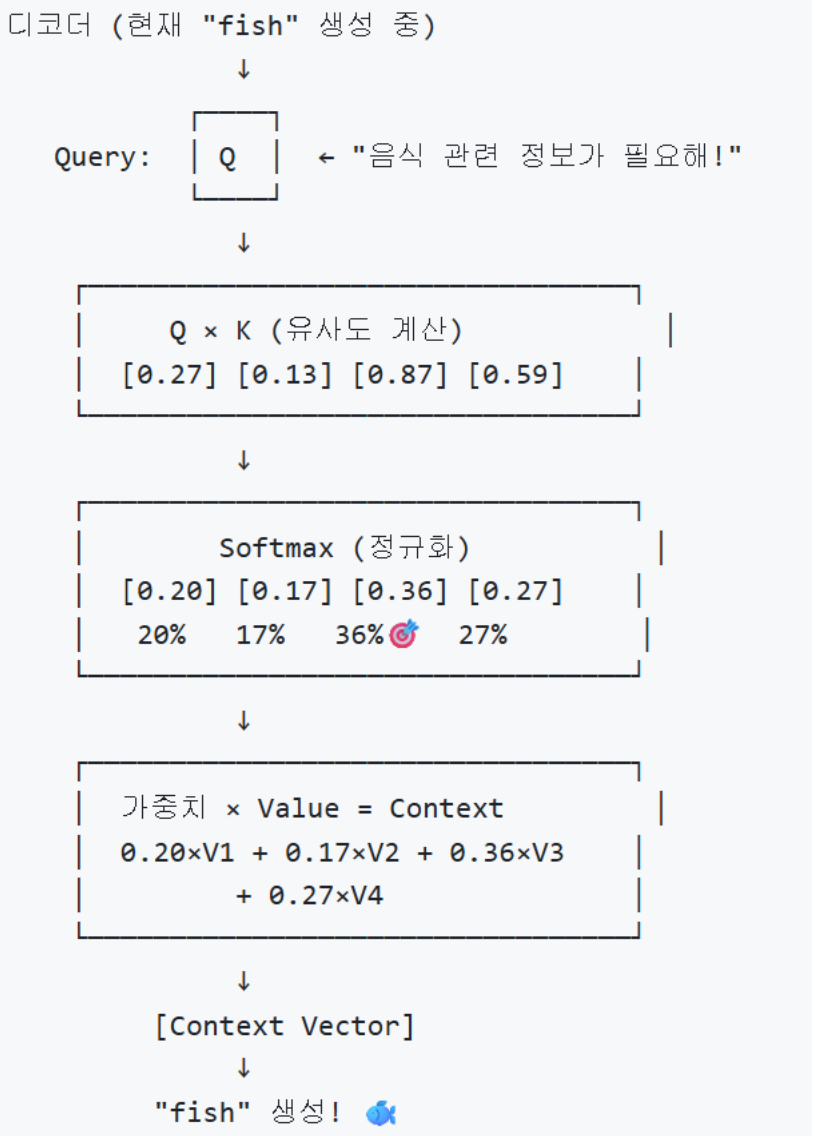
→ "철수가 뭘 찾고 있나?"
→ [축구친구찾기:0.9, 게임파티원찾기:0.8, 수학문제풀이친구:0.7]

Key (열쇠) 관점:

→ "철수가 뭘 제공할 수 있나?"
→ [축구잘함:0.9, 게임잘함:0.8, 수학잘함:0.9]

Value (가치) 관점:

→ "철수의 실제 능력은?"
→ {축구실력, 게임스킬, 수학지식, 성격, 경험 등 풍부한 정보}



트랜스포머

병렬 처리로 학습 속도를 극대화하면서 **장거리 의존성을 효과적으로 학습**하는게 목적
순환신경망을 제거하고 **어텐션 메커니즘**만으로 순서가 있는 데이터를 처리("Attention is All You Need" 논문 by 구글)

병렬처리

모든 단어를 차례대로가 아닌 동시에 처리하여
학습 속도를 극대화

Self-Attention

문장의 모든 단어가 서로의 관계를 파악하여
문맥 이해

Multi-Head Attention

복수개의 서로 다른 관점으로 동시에
문장을 분석(예: 주어-동사 관계, 순서관계 등)

잔차연결(Residual Connection)

원본 정보를 계속 보존하면서 새 정보를 누적
* Add & Norm: 잔차처리(잔차연결 + 정규화)

Masked Self-Attention

정답 출력 시 아직 생성 안된 미래 단어는
참조 금지 시킴

ENCODER

1 병렬 처리

모든 단어 동시 처리 → 속도 극대화

2 Self-Attention

모든 단어 쌍의 관계 파악 → 문맥 이해

3 Multi-Head Attention

여러 관점으로 동시 분석

4 잔차 연결 + Add&Norm

원본 보존 + 새 정보 누적

7 Feed Forward Network

비선형 변환 → 표현력 증가

×N 반복

DECODER

6 Masked Self-Attention

미래 단어 참조 금지 (순차 생성)

5 Cross-Attention

인코더 출력 지속 참조 → 정확한 생성

3 Multi-Head Attention

다양한 관점 유지

4 잔차 연결 + Add&Norm

정보 흐름 유지

7 Feed Forward Network

비선형 변환 → 표현력 증가

×N 반복

Cross-Attention

정답 출력 시 생성되는 각 단어가 입력 정보를
지속적으로 참조

Feed Forward Network

각 단어를 독립적으로 차원 확장 → 비선형 변환
→ 차원 축소하여 복잡한 패턴 학습

Attention이 단어 '간' 관계라면, FFN은 각 단어 '내' 심층 분석

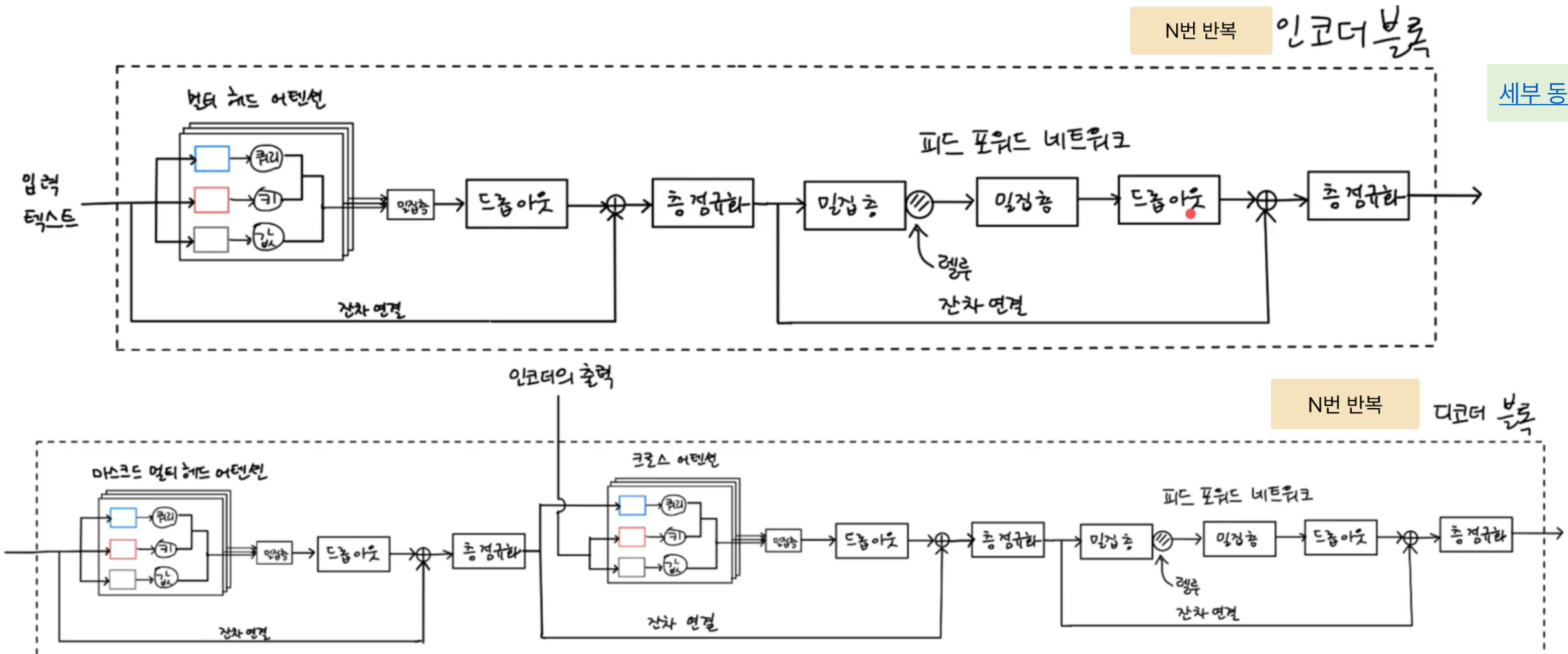
트랜스포머 전체 흐름

Self/Masked/Cross-Attention과 FFN이 핵심

암기법: 셀프(S) 마(M)크(C) 중인 피파(FF)

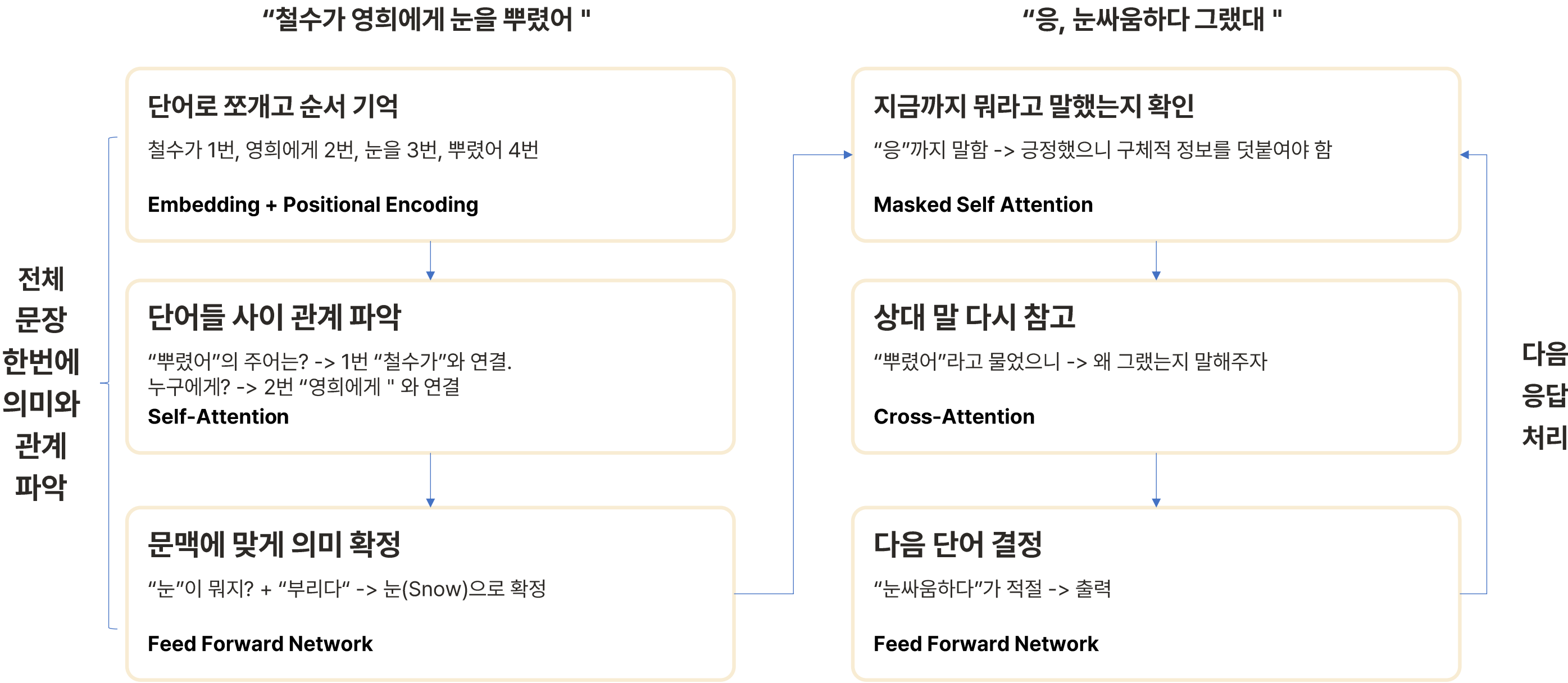
- ① 입력 단어 분할 및 순서 기억: Embedding, Positional Encoding
- ② 여러 관점으로 단어 간 관계 파악: **Multi-Head Self-Attention**
- ③ 문맥에 맞게 의미 확정: **Feed Forward Network**

- ④ 지금까지 응답 확인: Multi-Head **Masked Self-Attention**
- ⑤ 여러 관점으로 원문 다시 보기: Multi-Head **Cross-Attention**
- ⑥ 깊게 다음 단어 고민: **Feed Forward Network**

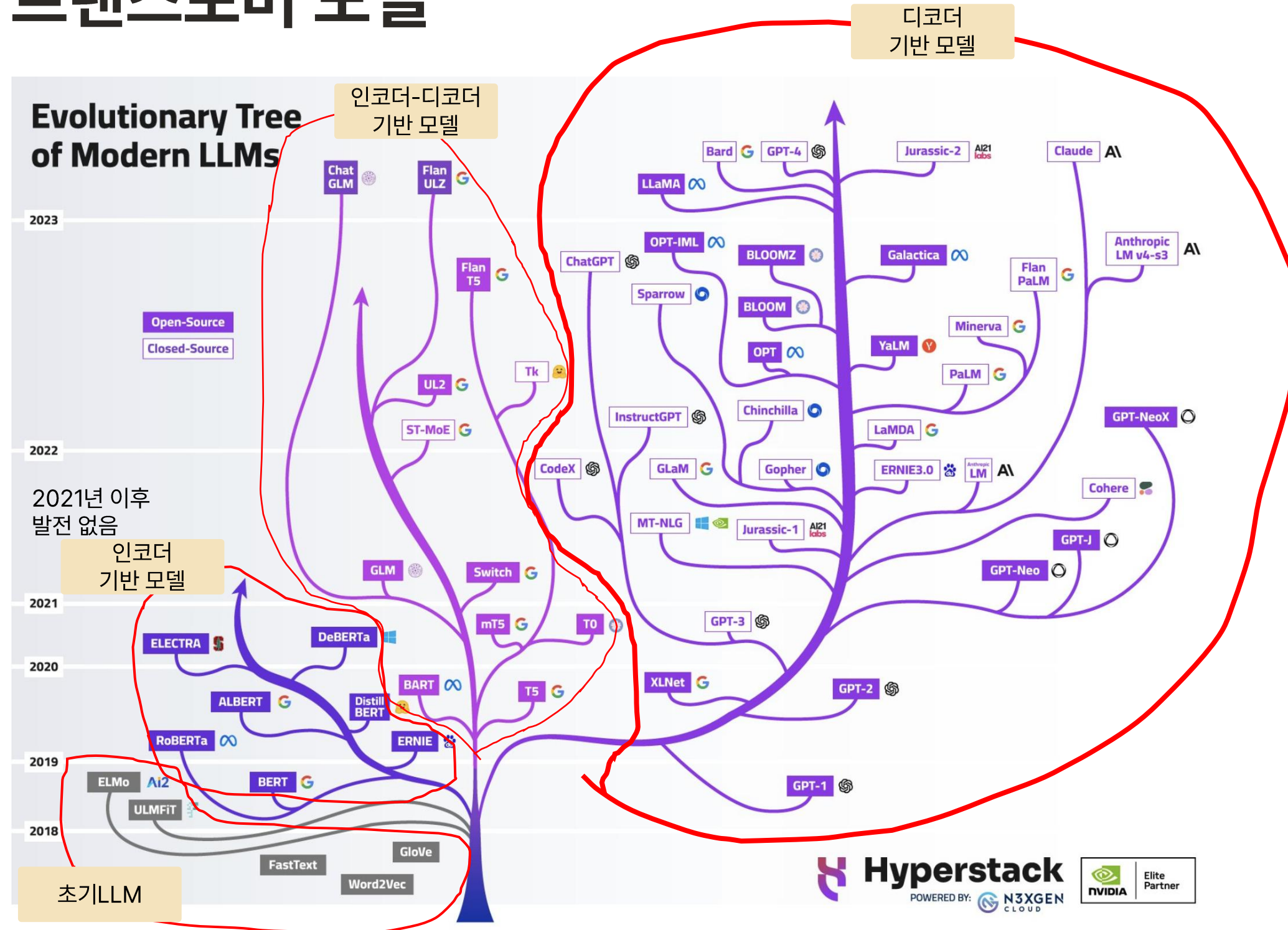


트랜스포머 전체 흐름

트랜스포머 모델은 사람이 상대방을 말을 이해하고 한 단어씩 최적의 답변을 하는 것과 동일



트랜스포머 모델



인코더-디코더 기반 모델

특징: 입력 텍스트를 이해하고 다른 형태의 출력 텍스트를 생성

적합한 작업: 번역, 요약에 강점

대표모델: T5, BART

디코더 기반 모델

특징:

- 충분히 큰 모델과 데이터로 인코더 없이 "단방향만으로도 문맥 파악" 가능
- 입력 프롬프트도 이전에 쓴 글 처럼 취급

적합한 작업: 텍스트 생성, 대화 시스템, 코드 생성, Q&A

대표모델: GTP시리즈, LLaMA, Claude

대표 오픈소스 모델(2025 현재)

- 라마3(Llama 3): 균형 잡힌 올라운더
- 젬마2(Gemma2): 효율성 + 안정성
- 관2(Qwen 2): 한국어 최강
- 파이-3(Phi-3): 소형 모델 특화
- 그래닛(Granite): 기업용 안전성 특화

전이학습

이미 훈련된 모델을 새로운 작업에 맞춰 재사용하거나 약간 조정(Fine Tuning)하여 사용하는 방법

사전 훈련 모델 다운로드 사이트

🤗 Hugging Face Hub

huggingface.co

가장 인기 있는 AI 모델 허브. 50만+ 모델, 25만+ 데이터셋 제공. NLP, Vision, Audio, Multimodal 모든 분야 지원.

BERT GPT-2 LLaMA
Stable Diffusion Whisper

💎 TensorFlow Hub

tfhub.dev

Google 공식 허브. TensorFlow 생태계에 최적화. 이미지, 텍스트, 비디오 모델 제공.

EfficientNet MobileNet BERT
Universal Sentence Encoder

🔥 PyTorch Hub

pytorch.org/hub

PyTorch 공식 허브. 연구용 최신 모델 빠른 배포. torch.hub.load()로 간편 사용.

ResNet YOLO DeepLab
Tacotron2

🦁 Model Zoo

modelzoo.co

다양한 프레임워크의 모델을 한 곳에서. 카테고리별 정리가 잘 되어 있음.

다양한 프레임워크 벤치마크 비교

❤️ NVIDIA NGC

catalog.ngc.nvidia.com

NVIDIA GPU 최적화 모델. 의료, 자율주행 등 산업 특화 모델.

Clara (의료) DRIVE (자율주행)
Megatron

📄 Papers with Code

paperswithcode.com

논문 + 코드 + 모델 + 벤치마크 통합. SOTA 모델 리더보드 제공.

SOTA 리더보드 벤치마크 논문 링크

디코더 기반 모델 Fine Tuning

입력층: 임베딩 + 위치 인코딩
단어를 벡터로 변환(WTE)하고
위치 정보(WPE) 추가
일반적으로 동결하여 사전학습된 어휘 표현 유지

디코더 블록 하위층: 1~6
기본적인 텍스트 생성 패턴 학습
GPT계열에서는 보통 동결함

디코더 블록 상위층: 7~12
스타일, 톤, 도메인 특화 생성을 담당
미세조정의 핵심 대상임(학습률이 제일 중요)

출력층: 언어 모델 헤드
다음 토큰 예측을 위한 출력층
보통 동결함. 단, 어휘사전 크기가 다르면 교체함

```
for param in model.transformer.wte.parameters():  
    param.requires_grad = False  
for param in model.transformer.wpe.parameters():  
    param.requires_grad = False
```

```
for name, param in model.named_parameters():  
    if "transformer.h" in name:  
        layer_num = int(name.split(".")[2])  
        if layer_num < 6: # 0-5번 블록 동결  
            param.requires_grad = False
```

```
training_args = TrainingArguments(  
    output_dir="./results",  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    learning_rate=2e-5,  
    warmup_steps=500,  
    weight_decay=0.01,  
)
```

```
for param in model.lm_head.parameters():  
    param.requires_grad = False
```

Domain-Adaptive Pre-training

범용 사전훈련 모델을 특정 도메인의 텍스트로 추가 사전훈련하여 해당 도메인에 적응시키는 기법

목적

특정 도메인의 특화된 지식, 용어, 문제를 학습

레이블 비용 절감:

- 분류, 감성분석, 개체명 인식, 질의응답은 레이블링 통한 지도학습 필요
- 레이블링은 높은 비용 필요하므로 DAPT로 자기지도학습하고 소량의 레이블로 지도학습을 함

방법

인코딩-디코딩 기반 모델(예:BERT)은 MLM(Masked Language Modeling) 사용

디코딩 기반 모델(예:GPT)은 CLM(Causal Language Modeling) 사용

특징	MLM (BERT)	CLM (GPT)
문맥 참조	양방향 (앞뒤 모두)	단방향 (앞만)
학습 방식	15% 토큰 마스킹 후 예측	다음 토큰 연속 예측
강점	문맥 이해, 분류	텍스트 생성
약점	생성 어려움	양방향 문맥 못 봄
대표 태스크	분류, NER, QA	대화, 글쓰기, 코딩

원본: "고객님의 주문이 발송되었습니다"

원문에 정답이 있으므로 레이블링 없이 학습 가능

◦ MLM 방식 (BERT)

입력: "고객님의 [MASK]이 발송되었습니다"

정답: "주문"

입력: "고객님의 주문이 [MASK]되었습니다"

정답: "발송"

입력: "[MASK]의 주문이 발송되었습니다"

정답: "고객님"

💡 무작위 위치를 마스킹하여 여러 번 학습

◦ CLM 방식 (GPT)

입력: "고객님의" → 정답: "주문이"

입력: "고객님의 주문이" → 정답: "발송"

입력: "고객님의 주문이 발송" → 정답: "되었습니다"

💡 한 문장에서 순차적으로 여러 학습 샘플 생성

토큰화 알고리즘

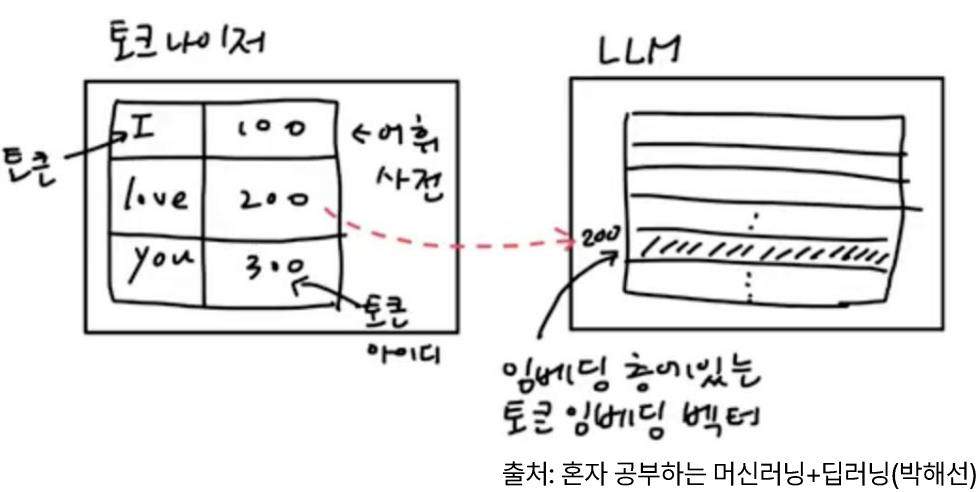
구분	BPE	WordPiece	Unigram	SentencePiece
정의	가장 빈번한 문자쌍을 반복 병합	가장 가깝게 등장하는 문자쌍을 반복 병합	손실이 적은 토큰을 반복 제거	공백을 특수문자로 치환하여 공백 구분없이 문자열 전체를 직접 토큰화
학습 방향	Bottom-up (병합)	Bottom-up (병합)	Top-down (제거)	BPE 또는 Unigram 선택
핵심 기준	빈도	우도(likelihood, 尤度)	손실(loss)	선택한 알고리즘에 따름
특징	단순, 구현 쉬움	언어 모델에 최적화	확률 기반 유연한 분할	다국어, 전처리 불필요
대표 모델	GPT-2, GPT-3	BERT, Electra	XLNet, ALBERT	T5, LLaMA
활용 케이스	- 영어 중심 텍스트 - 코드 생성 (Codex, Copilot) - 빠른 프로토타이핑 - 단순한 파이프라인	- 텍스트 분류/감성 분석 - 질의응답 - 명명된 개체 인식 (NER) - 문장 유사도 측정	- 형태소 복잡한 언어 (한국어, 일본어, 터키어) - 학습 시 토큰 분할을 다양하게 변형 필요 시	- 다국어/번역 모델 - 학습데이터가 적은 언어(몽골어 등) - 공백 없는 언어 (중국어, 태국어 등) - 코드+자연어 혼합 데이터



알고리즘	핵심 아이디어
BPE	"자주 붙어 나오면 하나로 합치자"
WordPiece	"합쳤을 때 의미가 커지면 합치자"
Unigram	"없어도 별 손해 없으면 빠자"
SentencePiece	"공백 구분 없이 통째로 학습하자"

토큰라이저와 임베딩 벡터

- 토큰라이저(Tokenizer)는 **각 토큰이** 어휘 사전에서 몇 번째 위치인지를 나타내는 **토큰 아이디로 변환**하는 역할. **모델과 별도로 존재**
- 토큰을 임베딩하는 역할은 LLM 모델의 임베딩 층이 담당함



* 우도(尤度: Likelihood): 어떤 사건이 일어날 가능성의 정도

디코딩 기반 모델 최신 기법

멀티쿼리 어텐션(MQA)

배경: 표준 MHA(Multi-Head Attention)는 각 헤드마다 독립적 Query, Key, Value 가중치 행렬을 가져야 해서 헤드 수만큼 메모리 증가

MQA: Q가중치 행렬은 헤드별 유지, K/V 가중치 행렬은 모든 헤드가 공유

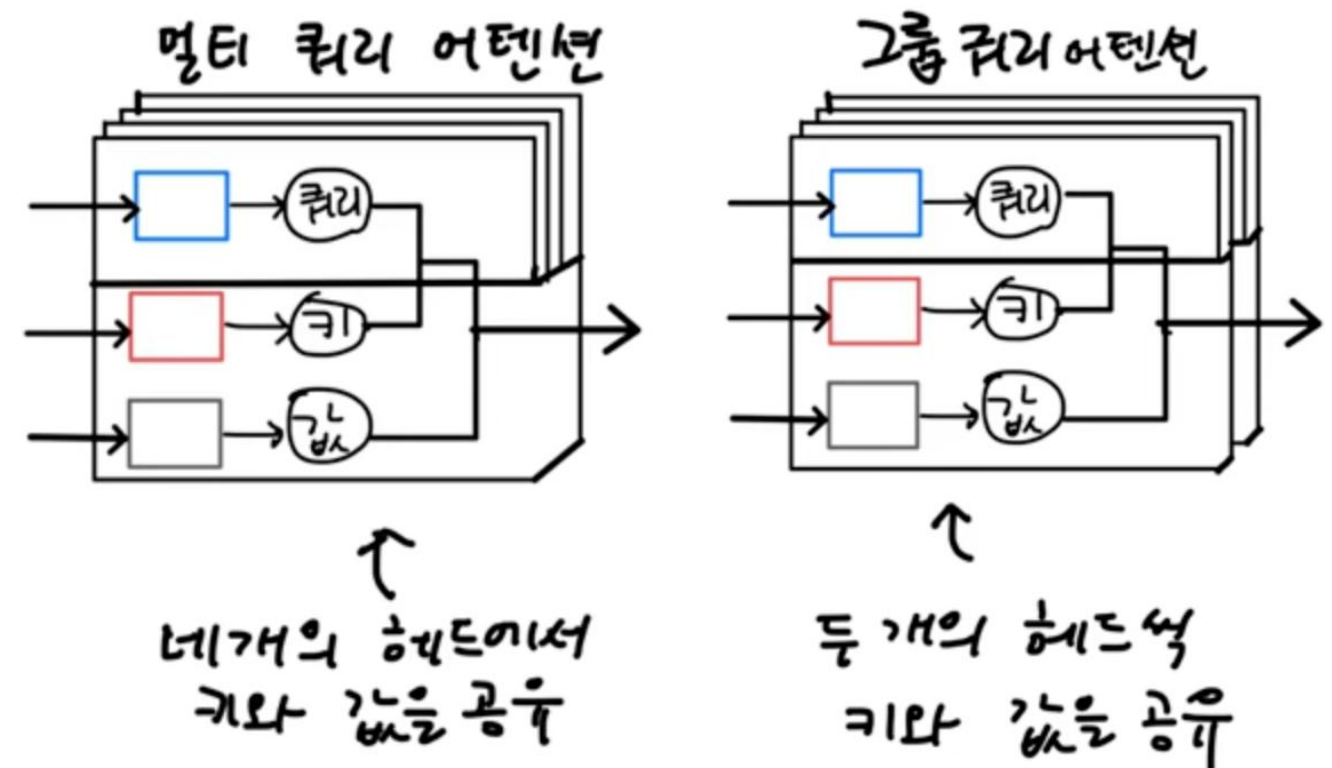
장점: 메모리 절약, 추론 속도 향상

단점: 품질 저하 가능성

그룹쿼리 어텐션(GQA)

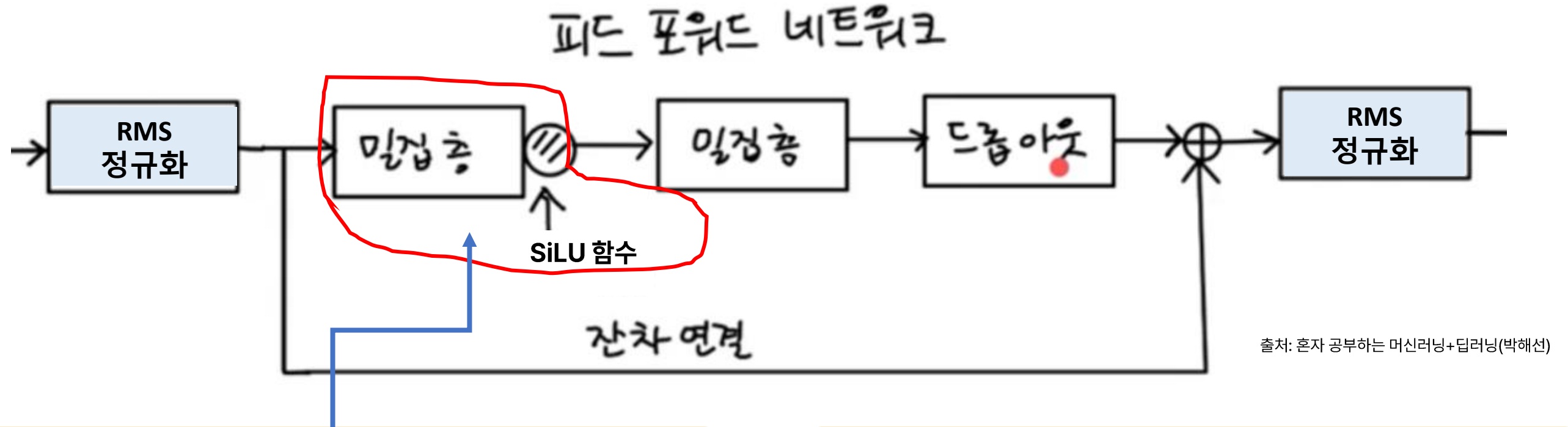
헤드를 N개의 그룹으로 분할. Q 가중치 행렬은 헤드별 유지, K/V 가중치 행렬은 각 그룹별로 공유

최신 오픈소스 모델은 GQA를 채택



출처: 혼자 공부하는 머신러닝+딥러닝(박해선)

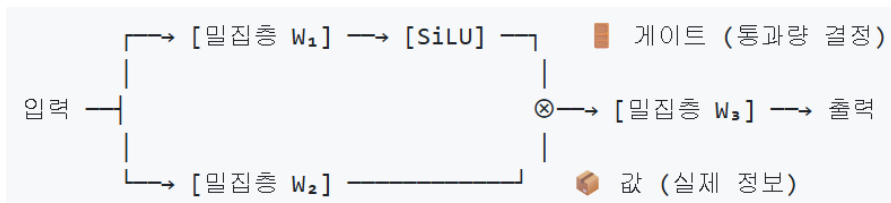
디코딩 기반 모델 최신 기법



SwiGLU 활성화

중요한 정보만 골라서 통과시켜 더 나은 성능을 제공하는 활성화 기법

- 밀집층 분할: 정보 통과량을 결정하는 게이트와 실제 정보로 나눔
- 게이트에 SiLU 활성화 함수 사용하여 학습 안정성 향상



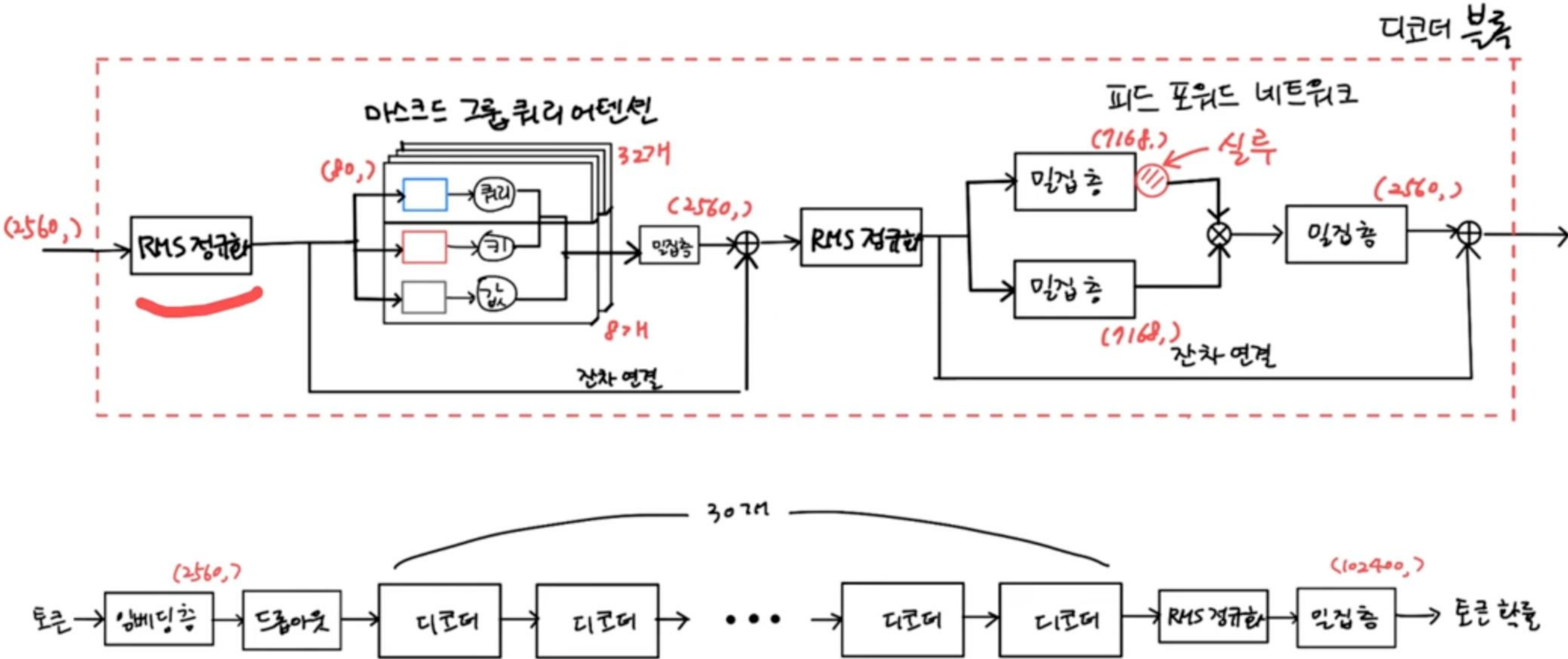
RMS(Root Mean Square) 정규화

층 정규화(Layer Normalization) 대비 계산을 단순화하여 학습 속도와 안정화를 향상시킨 정규화 기법

- 평균 계산을 생략하여 10~15% 연산량 절감
- 레이어 출력의 스케일을 일정하게 유지하여 학습 안정성 향상

디코딩 기반 모델 최신 기법

EXAONE-3.5 구성도



엑사원(EXAONE)은 **LG AI 연구원**이 개발한 디코딩 기반 멀티모달 LLM시리즈로 **한국어와 영어에 최적화된** 대규모 모델임

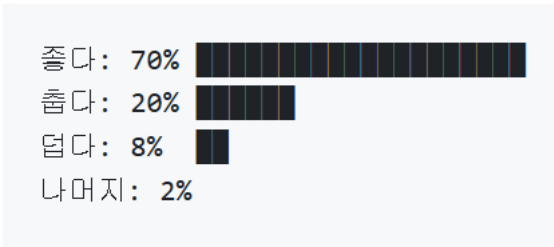
샘플링 전략

출력의 다양성과 일관성을 조절하는 파라미터

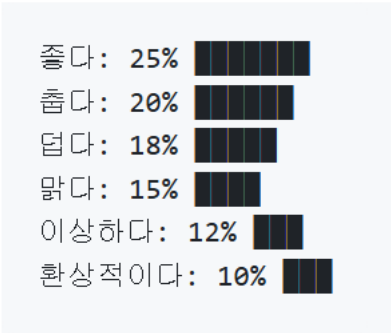
1. 정의

파라미터	정의	값 범위	효과
Temperature	확률 분포의 평탄화 정도 조절	0 ~ 2	낮으면 고확률 토큰 집중, 높으면 저확률 토큰도 선택 가능
Top-K	상위 K개 토큰만 후보로 고정	1 ~ 전체	후보 수 고정으로 극단적 토큰 제외 (현대 API에서 잘 사용 안 함)
Top-P	누적 확률 P까지 동적 후보 선정	0 ~ 1	맥락에 따라 후보 수 자동 조절, Top-K보다 유연함

Temperature = 0.2
예측 가능하고 일관된 출력



Temperature = 1.5
다양한 출력



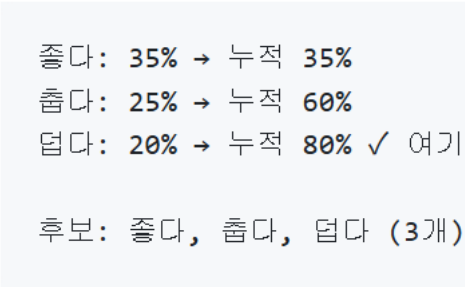
2. 용도별 적절 파라미터값

용도	Temperature	Top-P	Top-K	비고
코드 생성	0.2	0.9	-	정확성 우선
일반 대화	0.7	0.95	-	균형잡힌 응답
창작 글쓰기	1.0~1.2	0.95	-	다양성 확보
브레인스토밍	1.3	1.0	100	극단적 아이디어 허용

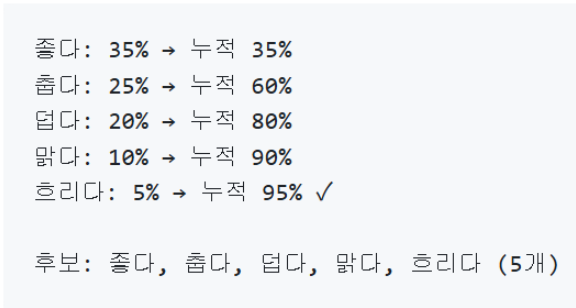
Top-K값이 "-"인 이유

- 현대 LLM API에서는 Top-P가 Top-K 보다 유연하여 기본 옵션으로 채택됨
- Top-K는 특수 용도나 연구 목적 외엔 조정 불필요

Top-P = 0.8 (80%)



Top-P = 0.95 (95%)



Q&A

